UNIVERSIDAD TECNOLÓGICA NACIONAL

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

BASE DE DATOS II

UNIDAD 6 · SEMANA 10

Mantenimiento de una Base de Datos

Respaldo y recuperación · Tareas programadas · Monitoreo

Aplicado al proyecto integrador Food Store

Ficha	
Asignatura	Base de Datos II, Unidad 6: Mantenimiento de una Base de Datos
Ubicación	Semana 10 de la cursada · 4 horas (2 h teoría + 2 h laboratorio)
Motor	PostgreSQL 16 o superior
Proyecto	Food Store: plan de mantenimiento de la capa de datos
Meta de la unidad	Respaldar, restaurar con evidencia, automatizar el mantenimiento rutinario y monitorear la salud de la base del proyecto, con IA bajo especificación y verificación humana
Ciclo lectivo	2026

Apunte teórico de cátedra · Material para el aula virtual.

Contenido

Presentación de la unidad

En las cinco unidades anteriores, la base de datos de Food Store se volvió consistente (Unidad 1), rápida (Unidad 2), ordenada en índices, vistas y objetos programables (Unidad 3), libre de redundancias no controladas (Unidad 4) y segura frente a accesos indebidos (Unidad 5). Todo ese trabajo tiene una condición de continuidad que todavía no se resolvió: **la base tiene que seguir funcionando mañana, el mes próximo y el año próximo**, aunque se rompa un disco, aunque alguien ejecute un `UPDATE` sin `WHERE` o aunque las tablas crezcan hasta que las consultas dejen de responder.

La Unidad 6 se ocupa de esa continuidad. Mantener una base de datos significa tres responsabilidades concretas, que dan nombre a las tres partes centrales de este apunte:

1. **Respaldo y recuperación:** poder volver a un estado correcto de los datos, en un tiempo conocido y probado.
2. **Tareas programadas de mantenimiento:** ejecutar en forma rutinaria, automática y registrada lo que el motor necesita para no degradarse (limpieza de versiones muertas, estadísticas, índices, archivado, verificación de integridad).
3. **Monitoreo:** medir la salud de la base con métricas y umbrales, para detectar un problema antes de que lo detecte el usuario final.

La unidad se dicta en la **semana 10** de la cursada, en un encuentro de cuatro horas: un bloque teórico-conceptual de dos horas (secciones 1 a 6 de este apunte) y un bloque de laboratorio de dos horas, en el que se ejecutan los scripts de las secciones 2 a 5 sobre la base del proyecto. La consigna del trabajo práctico de laboratorio se entrega en un documento aparte.

Dato	Detalle
Unidad	6 · Mantenimiento de una Base de Datos
Ubicación	Semana 10 de la cursada (4 horas: 2 h teoría + 2 h laboratorio)
Motor de referencia	PostgreSQL 16 (las diferencias con la versión 17 se indican en cada caso)
Proyecto	Food Store: plan de mantenimiento de la capa de datos
Prerrequisitos	Esquema `schema_completo.sql` cargado y poblado con `Genera_registros.sql`; roles de la Unidad 5 creados
Herramientas de IA	OpenCode (generación y revisión de scripts) y Kiro (especificación)
Producto de la unidad	Carpeta `mantenimiento/` del repositorio con scripts probados, agenda programada y bitácora

Objetivos de aprendizaje

Al finalizar la unidad, cada estudiante estará en condiciones de:

- Definir los objetivos de recuperación (RPO y RTO) de una base y elegir la estrategia de respaldo que los cumple.

- Ejecutar respaldos lógicos y físicos con `pg_dump`, `pg_dumpall` y `pg_basebackup`, y restaurar en forma completa, selectiva y a un instante determinado (PITR).
- Demostrar con evidencia que un respaldo se puede restaurar, midiendo el tiempo de restauración.
- Explicar por qué PostgreSQL genera tuplas muertas y cuándo corresponde `VACUUM`, `VACUUM FULL`, `ANALYZE` y `REINDEX`.
- Programar tareas rutinarias con el planificador del sistema operativo y con `pg_cron`, dejando registro en una bitácora.
- Construir un tablero de métricas de salud con umbrales y detectar problemas en forma temprana.
- Generar scripts de mantenimiento con IA a partir de una especificación, leerlos línea por línea y probarlos en un entorno descartable antes de programarlos.

> **Sobre el trabajo con inteligencia artificial en esta unidad**
>
> Un script de mantenimiento se ejecuta sin supervisión, de madrugada y con privilegios elevados. Por esa razón, un error en un script de mantenimiento tiene un alcance mayor que un error en una consulta: se repite todos los días hasta que alguien lo detecta. La regla de la cátedra para esta unidad es la siguiente: **la IA redacta el script; el estudiante lo lee completo, lo prueba en un entorno descartable, verifica su comportamiento ante una falla y recién entonces lo programa**. Un script que el estudiante no puede explicar línea por línea no se programa.

> **Cómo se construyó este apunte**
>
> Todos los comandos y consultas de este apunte se ejecutaron sobre una instancia de PostgreSQL 16.15 con la base Food Store poblada con el script de carga masiva del proyecto (50.000 productos, 20.000 usuarios, 200.000 pedidos y cerca de 500.000 líneas de detalle). Los bloques verdes titulados «Salida obtenida» reproducen la respuesta real del motor. En la computadora de cada estudiante los identificadores, los tamaños y los tiempos van a diferir levemente, porque los datos de prueba se generan con valores aleatorios.

1. El mantenimiento como responsabilidad profesional

1.1. Qué se degrada en una base que «funciona»

Una base de datos recién creada y bien diseñada no permanece en ese estado. Con el uso cotidiano, sin que nadie cometa un error, se producen los siguientes fenómenos:

Fenómeno	Causa	Consecuencia si no se atiende	Tarea que lo resuelve
Versiones muertas de filas (tuplas muertas)	Cada `UPDATE` y `DELETE` deja la versión anterior de la fila en la tabla (MVCC)	Las tablas y los índices crecen; las consultas leen más páginas	`VACUUM` (autovacuum)

Fenómeno	Causa	Consecuencia si no se atiende	Tarea que lo resuelve
Estadísticas desactualizadas	Cargas masivas o cambios en la distribución de los datos	El optimizador elige planes de ejecución malos (Unidad 2)	`ANALYZE`
Índices inflados o sin uso	Actualizaciones frecuentes; índices creados y luego olvidados	Espacio desperdiciado y escrituras más lentas (Unidad 3)	`REINDEX`, revisión de `pg_stat_user_indexes`
Consumo del contador de transacciones	Cada transacción consume un identificador (XID) de 32 bits	Riesgo de detención de la base para protegerse	`VACUUM` con congelamiento
Datos que ya no se consultan	Pedidos históricos, cancelados o dados de baja	Tablas operativas más grandes de lo necesario	Archivado planificado
Fallas y errores humanos	Disco dañado, borrado accidental, script defectuoso	Pérdida de datos	Respaldo y recuperación

El mantenimiento no es una tarea que se hace «cuando hay tiempo». Es un conjunto de procedimientos con frecuencia definida, responsable asignado y evidencia de ejecución. Esa es la diferencia entre una base que funciona por casualidad y una base administrada.

1.2. El ciclo de mantenimiento

Las tareas de la unidad se organizan en un ciclo que se repite en forma continua. Cada etapa produce información para la siguiente: el monitoreo detecta, por ejemplo, que una tabla acumula tuplas muertas; esa observación lleva a ajustar el autovacuum de esa tabla (mejora), que se vuelve a medir en el ciclo siguiente.

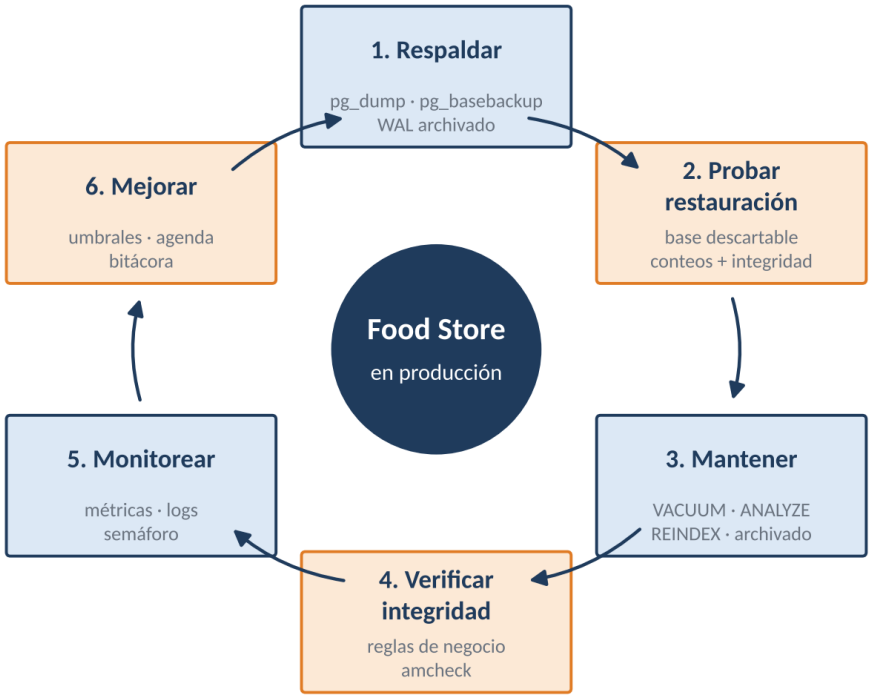


Figura 1. *Ciclo de mantenimiento de una base en producción. Cada etapa se desarrolla en una sección de este apunte.*

1.3. El caso de Food Store al comenzar la unidad

El caso de Food Store

Después de la carga masiva de la Unidad 2, la base del proyecto ocupa alrededor de **130 MB**. La tabla de mayor tamaño es `detalle_pedido`, seguida de `pedido`. La tabla siguiente muestra los tamaños medidos al comenzar la unidad: datos e índices por separado. Estas cifras son la línea de base contra la que se van a comparar los resultados del mantenimiento.

Consulta de tamaños por tabla

```
SELECT relname                                  AS tabla,
       pg_size_pretty(pg_table_size(relid))       AS datos,
       pg_size_pretty(pg_indexes_size(relid))     AS indices,
       pg_size_pretty(pg_total_relation_size(relid)) AS total
FROM   pg_stat_user_tables
WHERE  schemaname = 'public'
ORDER  BY pg_total_relation_size(relid) DESC;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
     tabla      |  datos   | indices |  total
----------------+----------+---------+---------
 detalle_pedido | 44 MB    | 26 MB   | 70 MB
 pedido         | 31 MB    | 8792 kB | 39 MB
 producto       | 6944 kB  | 1768 kB | 8712 kB
 usuario        | 2496 kB  | 1840 kB | 4336 kB
 categoria      | 40 kB    | 32 kB   | 72 kB
```

Para no mezclar los objetos de mantenimiento con los del dominio, la unidad crea un esquema separado, `mantenimiento`, que contiene la bitácora de ejecuciones, las tablas históricas para el archivado y las funciones de verificación y monitoreo. El script se aplica una sola vez, conectado como `admin_datos` (el dueño de los objetos definido en la Unidad 5):

Archivo 02_esquema_mantenimiento.sql

```
-- ============================================================
-- Unidad 6 · 02_esquema_mantenimiento.sql
-- Esquema separado para los objetos de mantenimiento de Food Store
-- ============================================================
CREATE SCHEMA IF NOT EXISTS mantenimiento;

-- 1) Bitácora: cada tarea programada deja constancia de su ejecución
CREATE TABLE mantenimiento.bitacora (
    id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tarea       VARCHAR(60)  NOT NULL,
    inicio      TIMESTAMPTZ  NOT NULL DEFAULT clock_timestamp(),
    fin         TIMESTAMPTZ,
    estado      VARCHAR(10)  NOT NULL DEFAULT 'EN_CURSO'
                CHECK (estado IN ('EN_CURSO','OK','ERROR')),
    filas       BIGINT,
    detalle     TEXT,
    ejecutado_por NAME NOT NULL DEFAULT current_user
);

-- 2) Tabla histórica para el archivado de pedidos antiguos
CREATE TABLE mantenimiento.pedido_historico
    (LIKE public.pedido INCLUDING DEFAULTS);
ALTER TABLE mantenimiento.pedido_historico
    ADD COLUMN archivado_en TIMESTAMPTZ NOT NULL DEFAULT now(),
    ADD PRIMARY KEY (id);
CREATE TABLE mantenimiento.detalle_pedido_historico
    (LIKE public.detalle_pedido INCLUDING DEFAULTS);
ALTER TABLE mantenimiento.detalle_pedido_historico
    ADD COLUMN archivado_en TIMESTAMPTZ NOT NULL DEFAULT now(),
    ADD PRIMARY KEY (id);
```

La bitácora es el corazón de la evidencia: cada tarea programada inserta una fila con su hora de inicio, su hora de fin, su estado (`OK` o `ERROR`) y un detalle. El tablero de monitoreo de la sección 5 consulta esta tabla para saber, por ejemplo, cuántas horas pasaron desde el último respaldo exitoso.

1.4. Roles de mínimo privilegio para el mantenimiento

La Unidad 5 estableció que cada identidad tiene exactamente los permisos que su función requiere. Las tareas de mantenimiento no son una excepción: el script de respaldo necesita **leer** todos los datos, pero no necesita modificarlos; el script de monitoreo necesita leer las **métricas** del motor, pero no los datos de negocio. PostgreSQL ofrece roles predefinidos que resuelven esas dos necesidades sin otorgar privilegios de superusuario:

Rol predefinido	Desde	Qué habilita
`pg_read_all_data`	PostgreSQL 14	SELECT sobre todas las tablas, vistas y secuencias, y USAGE sobre todos los esquemas. Suficiente para `pg_dump`.
`pg_monitor`	PostgreSQL 10	Leer todas las vistas de estadísticas y de actividad (`pg_stat_*`) y los parámetros de configuración.

Rol predefinido	Desde	Qué habilita
`pg_maintain`	PostgreSQL 17	Ejecutar `VACUUM`, `ANALYZE`, `REINDEX`, `CLUSTER` y `REFRESH MATERIALIZED VIEW` sobre cualquier tabla.

Archivo 01_roles_mantenimiento.sql

```
-- ============================================================
-- Unidad 6 · 01_roles_mantenimiento.sql
-- Roles de mínimo privilegio para las tareas de mantenimiento
-- ============================================================

-- Rol de grupo: puede LEER todo (para respaldar), no puede modificar nada
CREATE ROLE rol_respaldo NOLOGIN;
GRANT pg_read_all_data TO rol_respaldo;          -- rol predefinido (PostgreSQL 14+)

-- Rol de grupo: puede LEER métricas del motor, no los datos de negocio
CREATE ROLE rol_monitoreo NOLOGIN;
GRANT pg_monitor TO rol_monitoreo;               -- rol predefinido (PostgreSQL 10+)

-- VACUUM / ANALYZE / REINDEX: en PostgreSQL 16 solo puede ejecutarlos el
-- DUEÑO de cada tabla (o un superusuario). Las tareas de mantenimiento se
-- ejecutan entonces con el rol dueño del esquema (admin_datos).
-- En PostgreSQL 17+ existe el rol predefinido pg_maintain:
--   CREATE ROLE rol_mantenimiento NOLOGIN;
--   GRANT pg_maintain TO rol_mantenimiento;

-- Identidades de login que usan los scripts programados
CREATE ROLE mant_backup  LOGIN PASSWORD 'cambiar_backup'  CONNECTION LIMIT 2;
CREATE ROLE mant_monitor LOGIN PASSWORD 'cambiar_monitor' CONNECTION LIMIT 2;
GRANT rol_respaldo  TO mant_backup;
GRANT rol_monitoreo TO mant_monitor;
```

Verificado en el laboratorio de la cátedra

En la primera ejecución de este script sobre PostgreSQL 16 se incluyó la sentencia `GRANT pg_maintain TO rol_mantenimiento;`. El motor respondió `ERROR: role "pg_maintain" does not exist`, porque ese rol predefinido recién existe a partir de la versión 17. La documentación de la versión instalada es la referencia: un script generado por IA que «funcionaba» en otra versión del motor no está probado en la versión del proyecto.

Las contraseñas del script son de ejemplo. En el entorno real, las credenciales de los roles de login se almacenan en el archivo de contraseñas del usuario del sistema operativo que ejecuta los scripts (`~/.pgpass` en Linux, `%APPDATA%\postgresql\pgpass.conf` en Windows), con permisos de lectura exclusivos para ese usuario, y nunca dentro del script ni en el repositorio.

2. Respaldo y recuperación

2.1. Conceptos previos

Respaldo, réplica y redundancia no son lo mismo

Un **respaldo** (backup) es una copia de los datos tomada en un momento determinado, almacenada por separado, que permite reconstruir la base en ese estado. Dos mecanismos que suelen confundirse con el respaldo no lo reemplazan:

- **La redundancia de discos (RAID)** protege contra la falla física de un disco. Si alguien ejecuta `DELETE FROM pedido;`, el RAID borra los pedidos en todos los discos al mismo tiempo.
- **La réplica** (un segundo servidor que recibe los cambios del primario) protege contra la caída del servidor. Un `UPDATE` sin `WHERE` se replica en milisegundos: la réplica copia también el error.

Solamente un respaldo permite volver a un estado **anterior** al error. Por eso los tres mecanismos se complementan y ninguno sustituye al respaldo.

RPO y RTO: las dos preguntas que definen la estrategia

Antes de elegir una herramienta, la organización responde dos preguntas de negocio. Las respuestas se expresan con dos indicadores definidos en la guía de planificación de contingencias del NIST (Swanson et al., 2010):

- **RPO (Recovery Point Objective, objetivo de punto de recuperación):** la cantidad máxima de datos, medida en tiempo, que se acepta perder. Un RPO de 24 horas significa que se acepta perder los pedidos de un día completo.
- **RTO (Recovery Time Objective, objetivo de tiempo de recuperación):** el tiempo máximo aceptable entre la falla y el restablecimiento del servicio.

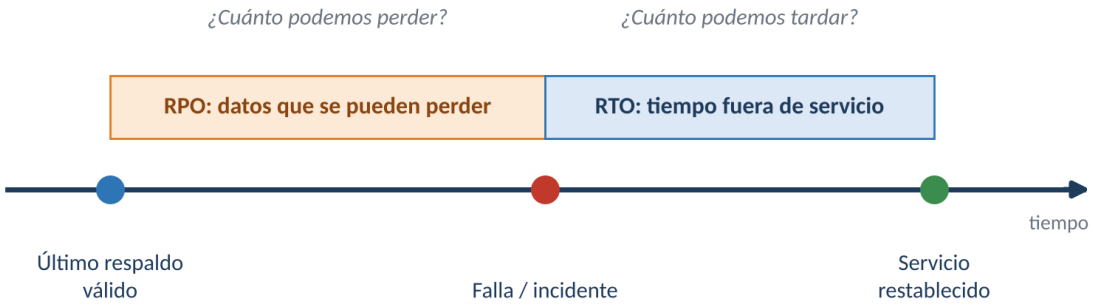


Figura 2. *RPO y RTO en la línea de tiempo de un incidente.*

> **El caso de Food Store: objetivos de recuperación**
>
> Food Store recibe pedidos durante todo el día y cada pedido confirmado representa una venta y un descuento de stock. Perder un día completo de pedidos obligaría a reconstruirlos a mano a partir de los comprobantes de pago. La cátedra fija para el proyecto los siguientes objetivos:

- **RPO = 5 minutos** para los datos operativos (pedidos y stock). Este valor no se puede cumplir con un respaldo diario: exige archivado continuo del registro de transacciones (WAL) y recuperación a un instante (sección 2.7).
- **RTO = 1 hora**. Este valor obliga a conocer, con una medición real, cuánto tarda la restauración (sección 2.8).
- Además, un **respaldo lógico diario**, que se puede restaurar en otra versión del motor y en forma selectiva por tabla.

2.2. Tipos de respaldo

Lógico y físico

PostgreSQL ofrece dos familias de respaldo, que difieren en **qué** se copia:

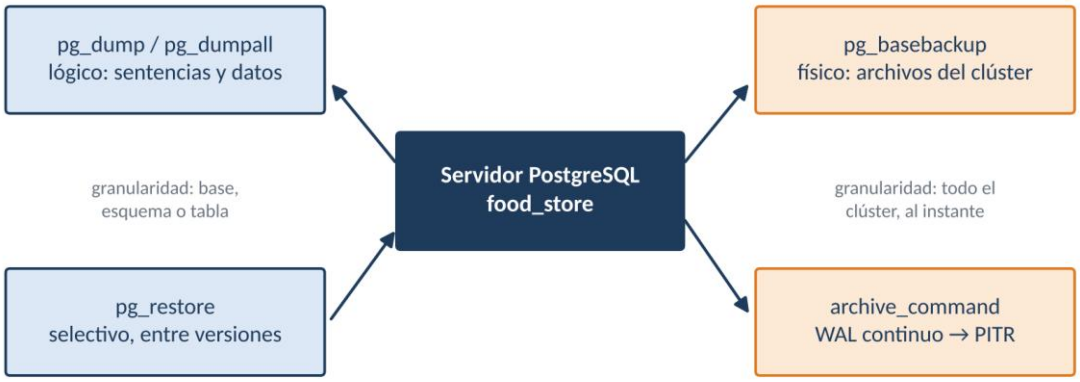


Figura 3. *Respaldo lógico y respaldo físico en PostgreSQL.*

Criterio	Respaldo lógico (pg_dump)	Respaldo físico (pg_basebackup + WAL)
Qué copia	Sentencias SQL y datos que reconstruyen los objetos	Los archivos de datos del clúster, bloque por bloque
Alcance	Una base, un esquema o una tabla	El clúster completo (todas las bases)
Consistencia	Instantánea transaccional al inicio del respaldo	Consistente al aplicar el WAL generado durante la copia
Restauración selectiva	Sí: una tabla, un esquema, solo la estructura	No: se restaura el clúster completo
Entre versiones del motor	Sí (se usa para migrar de versión)	No: misma versión mayor y misma plataforma
Recuperación a un instante (PITR)	No: solo al momento del respaldo	Sí, combinado con el archivado continuo de WAL
Velocidad en bases grandes	Lenta para restaurar (reconstruye índices)	Rápida (copia de archivos)

Completo, diferencial e incremental

Con independencia de la familia, un respaldo puede copiar todo o solamente lo que cambió:

- **Completo:** copia la totalidad de los datos. Se restaura con un único archivo, pero cada ejecución ocupa el 100 % del tamaño de la base.
- **Diferencial:** copia lo que cambió desde el último respaldo **completo**. Para restaurar se necesitan dos piezas: el completo y el último diferencial.
- **Incremental:** copia lo que cambió desde el último respaldo de **cualquier tipo**. Ocupa menos espacio, pero para restaurar se necesita el completo y toda la cadena de incrementales posteriores, sin que falte ninguno.

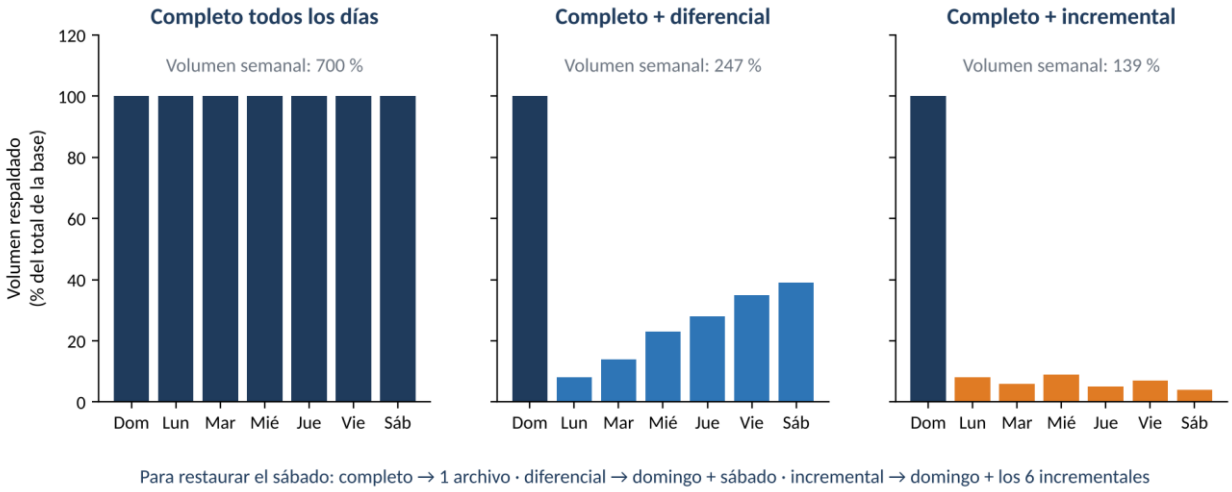


Figura 4. *Volumen respaldado en una semana con cada esquema (valores ilustrativos de cambio diario).*

En PostgreSQL, el archivado continuo de WAL cumple el papel de un respaldo incremental permanente: cada segmento de 16 MB del registro de transacciones contiene los cambios ocurridos desde el segmento anterior. A partir de la versión 17, además, `pg_basebackup --incremental` produce respaldos físicos incrementales que se combinan con la herramienta `pg_combinebackup`.

2.3. Dónde se guardan los respaldos: la regla 3-2-1

Un respaldo almacenado en el mismo disco que la base se pierde junto con la base. La recomendación conocida como regla 3-2-1, difundida por el equipo de respuesta a incidentes del gobierno de los Estados Unidos (Ruggiero y Heckathorn, 2012), establece tres condiciones:



Figura 5. *La regla 3-2-1 de almacenamiento de respaldos.*

Dos agregados actuales completan la regla. Primero, al menos una copia debe ser **inmutable** o estar desconectada, porque un ataque de ransomware cifra también los respaldos que encuentra accesibles. Segundo, los respaldos contienen los mismos datos personales que la base (en Food Store: nombre, apellido, correo y celular de cada cliente), de modo que están alcanzados por las mismas obligaciones de protección de la Ley 25.326 que se estudiaron en la Unidad 5: se almacenan cifrados y con acceso restringido.

2.4. Respaldo lógico con pg_dump

`pg_dump` toma una instantánea consistente de una base sin bloquear a los usuarios: abre una transacción de solo lectura y lee los datos tal como estaban al comienzo, aunque otras sesiones sigan escribiendo. Admite cuatro formatos de salida:

Formato	Opción	Características	Se restaura con
Texto plano	`-Fp`	Archivo .sql legible; no permite restauración selectiva ni en paralelo	`psql -f`
Personalizado (custom)	`-Fc`	Un archivo comprimido; restauración selectiva y en paralelo. **Formato recomendado**	`pg_restore`
Directorio	`-Fd`	Un archivo por tabla; permite respaldar en paralelo (`-j`)	`pg_restore`
Tar	`-Ft`	Archivo tar sin compresión; poco usado	`pg_restore`

Respaldo lógico en formato personalizado

```
# Respaldo de food_store con el rol de mínimo privilegio mant_backup
pg_dump -h localhost -U mant_backup -d food_store -Fc -Z 6 \
        -f food_store_20261005_0200.dump

# Inspeccionar el contenido sin restaurar: índice (TOC) del archivo
pg_restore --list food_store_20261005_0200.dump | head -30
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
;     dbname: food_store
;     TOC Entries: 68
;     Compression: gzip
;     Format: CUSTOM
```

```
;     Dumped from database version: 16.15
...
869; 1247 16392 TYPE public estado_pedido admin_datos
236; 1255 16491 FUNCTION public calcular_total_pedido(bigint) admin_datos
250; 1255 16497 PROCEDURE public sp_crear_pedido(bigint, public.forma_pago, jsonb) admin_datos
219; 1259 16410 TABLE public categoria admin_datos
...
3531; 0 16410 TABLE DATA public categoria admin_datos
3537; 0 16452 TABLE DATA public pedido admin_datos

real 0m2.093s     (tamaño del archivo: 12 MB para una base de 129 MB)
```

Tres observaciones sobre la salida. Primero, el respaldo comprimido ocupa alrededor del 10 % del tamaño de la base, porque no contiene los índices (solo la sentencia que los crea) y los datos se comprimen. Segundo, el respaldo incluye funciones, procedimientos, disparadores y vistas de las unidades anteriores: todo el esquema del proyecto queda respaldado. Tercero, el rol `mant_backup` pudo leer todas las tablas sin ser dueño ni superusuario, gracias a `pg_read_all_data`.

2.5. Lo que pg_dump no respalda

`pg_dump` respalda **una base**. Los roles, sus contraseñas y sus pertenencias son objetos **globales** del clúster: no pertenecen a ninguna base y, por lo tanto, no están en el archivo anterior. Si el servidor se pierde y se restaura solamente el respaldo de `food_store`, todo el esquema de seguridad de la Unidad 5 desaparece y los `GRANT` del respaldo fallan porque los roles no existen. Los objetos globales se respaldan con `pg_dumpall`:

Respaldo de los objetos globales

```
# Solo roles y tablespaces (objetos globales), sin las contraseñas
pg_dumpall -h localhost -U postgres --globals-only --no-role-passwords \
           -f food_store_globales_20261005.sql
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
CREATE ROLE admin_datos;
ALTER ROLE admin_datos WITH NOSUPERUSER INHERIT NOCREATEROLE CREATEDB LOGIN ...;
CREATE ROLE app_web;
ALTER ROLE app_web WITH NOSUPERUSER INHERIT ... LOGIN ... CONNECTION LIMIT 50;
CREATE ROLE mant_backup;
CREATE ROLE rol_respaldo;
...
GRANT pg_read_all_data TO rol_respaldo WITH INHERIT TRUE GRANTED BY postgres;
GRANT rol_app_escritura TO app_web WITH INHERIT TRUE GRANTED BY postgres;
GRANT rol_respaldo TO mant_backup WITH INHERIT TRUE GRANTED BY postgres;
```

Error frecuente

Respaldar solamente la base con `pg_dump` y descubrir, durante una restauración real, que faltan los roles. La opción `--no-role-passwords` evita que los hashes de las contraseñas queden en un archivo de texto; a cambio, después de restaurar los roles hay que volver a asignar las contraseñas desde el gestor de secretos de la organización. Esa decisión se documenta en el plan de mantenimiento.

2.6. Restauración: completa y selectiva

El formato personalizado permite elegir qué restaurar. Los casos más frecuentes son tres:

Variantes de restauración con pg_restore

```
# 1) Restauración completa en una base nueva, con 4 procesos en paralelo
createdb -h localhost -U postgres food_store_restaurada
pg_restore -h localhost -U postgres -d food_store_restaurada -j 4 \
           --exit-on-error food_store_20261005_0200.dump


# 2) Solo la estructura (tablas, funciones, vistas), sin datos
pg_restore -h localhost -U postgres -d food_store_vacia \
           --section=pre-data food_store_20261005_0200.dump


# 3) Solo los datos de una tabla
pg_restore -h localhost -U postgres -d food_store_vacia \
           --data-only -t categoria food_store_20261005_0200.dump


# 4) Restauración a medida: se edita el índice y se restaura lo que quedó en él
pg_restore --list food_store_20261005_0200.dump > indice.list
#   (se borran o se comentan con ';' las líneas que no se quieren restaurar)
pg_restore -h localhost -U postgres -d food_store_restaurada -L indice.list \
           food_store_20261005_0200.dump
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
-- Resultado de las variantes 2 y 3 sobre una base nueva:
SELECT count(*) FROM categoria;   -->  5
SELECT count(*) FROM producto;    -->  0     (la tabla existe, sin datos)
```

La opción `--exit-on-error` es importante en un procedimiento automatizado: por omisión, `pg_restore` informa los errores y **continúa**, de modo que una restauración incompleta puede pasar inadvertida. Con esta opción, el primer error detiene el proceso con código de salida distinto de cero, y el script que lo invoca se entera de la falla.

2.7. Respaldo físico y recuperación a un instante (PITR)

El registro de transacciones (WAL)

Antes de modificar un archivo de datos, PostgreSQL escribe el cambio en el **WAL** (Write-Ahead Log, registro de escritura anticipada). La Unidad 1 lo presentó como el mecanismo que garantiza la durabilidad de una transacción confirmada. Para el respaldo, el WAL tiene una segunda utilidad: si se conserva una copia de todos los segmentos del WAL generados desde un respaldo físico, es posible reconstruir la base en **cualquier instante** posterior a ese respaldo, reaplicando los cambios uno por uno hasta el momento elegido. Esta técnica se denomina **PITR** (Point-In-Time Recovery).

Configuración del archivado continuo

Parámetros del archivado continuo de WAL (Linux)

```
# postgresql.conf (requiere reiniciar el servicio la primera vez)
wal_level       = replica
archive_mode    = on
# %p = ruta del segmento a archivar; %f = nombre del archivo
# El comando falla (código distinto de 0) si el archivo ya existe:
# así nunca se sobrescribe un segmento archivado.
archive_command = 'test ! -f /respaldos/wal/%f && cp %p /respaldos/wal/%f'
```

En Windows, el comando equivalente es `archive_command = 'copy "%p" "D:\\respaldos\\wal\\%f"'`. En producción, la carpeta de destino está en otro equipo o en almacenamiento externo (regla 3-2-1).

Respaldo base con pg_basebackup

Respaldo físico y verificación

```
pg_basebackup -h localhost -U postgres -D /respaldos/base_20261006 \
              -Fp -X stream -c fast -P -l "base Food Store U6"


# Verificar la integridad del respaldo contra su manifiesto (PostgreSQL 13+)
pg_verifybackup /respaldos/base_20261006
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
269787/269787 kB (100%), 1/1 tablespace
backup successfully verified


-- Contenido del archivo backup_label generado:
START WAL LOCATION: 0/1C000028 (file 000000010000000000000001C)
BACKUP METHOD: streamed
START TIME: 2026-10-05 21:13:04 -03
LABEL: base Food Store U6
```

Ensayo: recuperar Food Store de un UPDATE sin WHERE

El ensayo siguiente se realizó sobre la base del proyecto y reproduce el incidente que la Unidad 1 presentó como el riesgo fundacional del trabajo con datos. Después del respaldo base, la aplicación registra un pedido nuevo; segundos después, un operador ejecuta un aumento de precios y olvida la cláusula `WHERE`.

Línea de tiempo del incidente

```
-- 21:13:10  Actividad normal posterior al respaldo base
CALL sp_crear_pedido(5, 'TARJETA',
     '[{"producto_id": 101, "cantidad": 2}, {"producto_id": 202, "cantidad": 1}]');


-- 21:13:13  El error
UPDATE producto SET precio = 0;   -- ¡sin WHERE!


-- Se fuerza el cierre del segmento WAL actual para que se archive
SELECT pg_switch_wal();
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
   id   |   fecha    |  estado   |  total  | usuario_id
--------+------------+-----------+---------+------------
 200014 | 2026-10-05 | PENDIENTE | 5108.16 |          5


       momento_del_error
-------------------------------
 2026-10-05 21:13:13.664907-03


UPDATE 50000
 productos_con_precio_cero
---------------------------
```

```
                      50000
```

La recuperación **no se hace sobre el servidor productivo**. Se levanta una instancia nueva a partir de una copia del respaldo base, con un puerto distinto, y se le indica hasta qué instante debe reaplicar el WAL:

Recuperación a un instante en una instancia separada

```
# 1) Copia del respaldo base en una carpeta nueva
cp -a /respaldos/base_20261006 /recuperacion/pitr


# 2) Parámetros de recuperación (se agregan a postgresql.auto.conf)
port                  = 5433
restore_command       = 'cp /respaldos/wal/%f %p'
recovery_target_time  = '2026-10-05 21:13:12-03'   # antes del error
recovery_target_action = 'promote'


# 3) Archivo vacío que indica al motor que debe iniciar en modo recuperación
touch /recuperacion/pitr/recovery.signal


# 4) Inicio de la instancia de recuperación
pg_ctl -D /recuperacion/pitr -l pitr.log start
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
LOG:  starting point-in-time recovery to 2026-10-05 21:13:12-03
LOG:  restored log file "00000001000000000000001C" from archive
LOG:  restored log file "00000001000000000000001D" from archive
LOG:  restored log file "00000001000000000000001E" from archive
LOG:  recovery stopping before commit of transaction 918, time 2026-10-05 21:13:13.89607-03
LOG:  selected new timeline ID: 2
LOG:  archive recovery complete
LOG:  database system is ready to accept connections

-- Verificación en la instancia recuperada (puerto 5433)
 productos_con_precio_cero | precio_promedio
---------------------------+-----------------
                         0 |         2747.89
   id   |  estado   |  total
--------+-----------+---------
 200014 | PENDIENTE | 5108.16
```

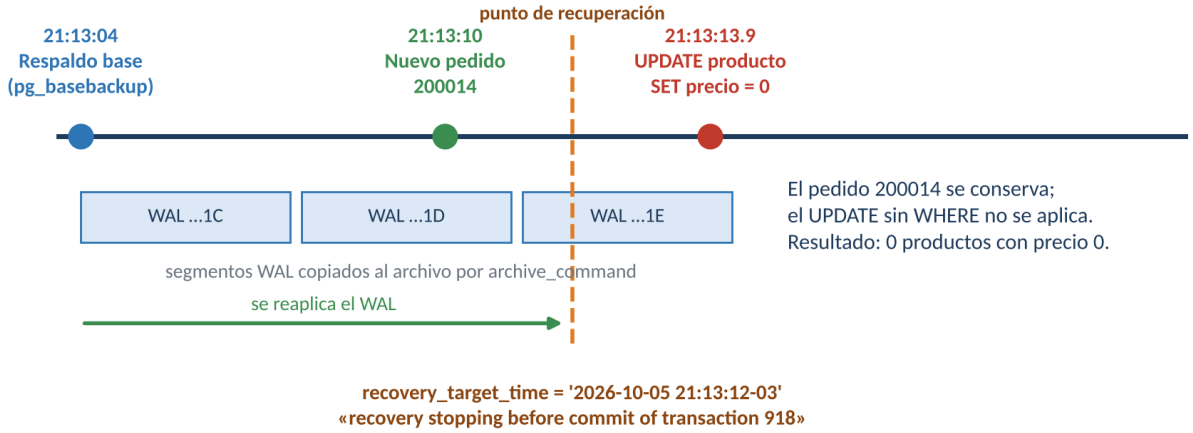


Figura 6. *Ensayo de PITR sobre Food Store: el WAL se reaplica hasta el instante anterior al error.*

El registro del motor muestra exactamente qué ocurrió: se reaplicaron tres segmentos del WAL y la recuperación se detuvo **antes de confirmar la transacción 918**, que era el `UPDATE` defectuoso. El pedido 200014, registrado tres segundos antes del error, se conservó. La nueva línea de tiempo (timeline 2) indica que, a partir de ese punto, la historia de la instancia recuperada se separó de la historia del servidor original.

Reparación quirúrgica del servidor productivo

Reemplazar la base productiva completa por la instancia recuperada descartaría todo lo que ocurrió después del error (pedidos legítimos registrados mientras se investigaba). Cuando el daño está acotado a una columna de una tabla, la práctica profesional es copiar desde la instancia recuperada **solamente los datos dañados**, dentro de una transacción y con un control antes de confirmar:

Reparación selectiva de la columna dañada

```
-- En la instancia recuperada (puerto 5433): exportar los precios correctos
\copy (SELECT id, precio FROM producto) TO 'precios_recuperados.csv' CSV HEADER

-- En el servidor productivo
BEGIN;
CREATE TEMP TABLE precios_ok (id BIGINT PRIMARY KEY, precio NUMERIC(10,2));
\copy precios_ok FROM 'precios_recuperados.csv' CSV HEADER
UPDATE producto p
SET    precio = o.precio
FROM   precios_ok o
WHERE  o.id = p.id
  AND  p.precio = 0;          -- solo las filas dañadas por el incidente
SELECT count(*) FILTER (WHERE precio = 0) AS con_precio_cero FROM producto;
COMMIT;                       -- solo si el control anterior devuelve 0
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
COPY 50000
UPDATE 50000
 con_precio_cero
-----------------
               0
COMMIT
```

2.8. La prueba de restauración como obligación profesional

Un archivo de respaldo cuya restauración nunca se ensayó es una hipótesis, no un respaldo. El capítulo sobre integridad de datos del libro de ingeniería de confiabilidad de Google lo resume así: «nadie quiere realmente hacer respaldos; lo que se quiere son restauraciones» (Blum y Singh, 2016). Los motivos por los que un respaldo aparentemente correcto no se puede restaurar son concretos: el archivo quedó truncado porque el disco se llenó, faltan los roles globales, una extensión no está instalada en el servidor de destino, la clave de cifrado se perdió o nadie sabe qué comando ejecutar.

La prueba de restauración se automatiza y se programa con la misma frecuencia con que se revisa el plan (semanal en Food Store). El script siguiente restaura el último respaldo en una base descartable, compara conteos de filas y la suma de importes contra la base de origen, ejecuta los controles de integridad de la sección 4.9 y **mide el tiempo de restauración**, que es el dato real con el que se valida el RTO:

Archivo prueba_restauracion.sh

```
#!/usr/bin/env bash
# ============================================================
# Unidad 6 · prueba_restauracion.sh
# Restaura el último respaldo en una base DESCARTABLE y lo compara
# con producción. Un respaldo que no se restauró no está probado.
# Se ejecuta en el servidor de PRUEBAS con su superusuario (crea
# extensiones y asigna dueños), nunca contra el servidor productivo.
# ============================================================
set -euo pipefail
PGHOST="${PGHOST:-localhost}"
PGUSER="${PGUSER:-postgres}"
DESTINO="${DESTINO:-/var/backups/food_store}"
PRUEBA="food_store_prueba_restauracion"

ULTIMO="$(ls -1t "$DESTINO"/food_store_*.dump | head -n 1)"
echo "Respaldo a probar: $ULTIMO"
sha256sum --check "${ULTIMO}.sha256"

INICIO=$(date +%s)
dropdb   -h "$PGHOST" -U "$PGUSER" --if-exists "$PRUEBA"
createdb -h "$PGHOST" -U "$PGUSER" "$PRUEBA"
# Índice (TOC) del respaldo sin pg_cron: esa extensión solo puede existir
# en la base configurada en cron.database_name (la productiva)
pg_restore --list "$ULTIMO" | grep -v -E 'pg_cron|SCHEMA cron' > /tmp/toc_prueba.list
pg_restore -h "$PGHOST" -U "$PGUSER" -d "$PRUEBA" --exit-on-error -j 4 \
           -L /tmp/toc_prueba.list "$ULTIMO"
FIN=$(date +%s)

CONTROL="SELECT 'categoria' t, count(*) FROM categoria UNION ALL
         SELECT 'producto', count(*) FROM producto UNION ALL
         SELECT 'usuario', count(*) FROM usuario UNION ALL
         SELECT 'pedido', count(*) FROM pedido UNION ALL
         SELECT 'detalle_pedido', count(*) FROM detalle_pedido UNION ALL
         SELECT 'suma_totales', sum(total)::bigint FROM pedido ORDER BY 1"

diff <(psql -h "$PGHOST" -U "$PGUSER" -d "$PRUEBA"     -XAtc "$CONTROL") \
     <(psql -h "$PGHOST" -U "$PGUSER" -d food_store    -XAtc "$CONTROL") \
  && echo "RESTAURACIÓN VERIFICADA: conteos idénticos. Tiempo de restauración: $((FIN-INICIO)) s" \
  || echo "ATENCIÓN: hay diferencias (normal si hubo actividad posterior al respaldo)"

psql -h "$PGHOST" -U "$PGUSER" -d "$PRUEBA" -XAtc \
     "SELECT control, resultado FROM mantenimiento.fn_verificar_integridad()"
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
Respaldo a probar: /var/lib/pgdemo/backups/food_store_20261006_0017.dump
/var/lib/pgdemo/backups/food_store_20261006_0017.dump: OK
RESTAURACIÓN VERIFICADA: conteos idénticos. Tiempo de restauración: 1 s
C1 total del pedido = suma de subtotales|OK
C2 subtotal = cantidad x precio_unitario|OK
C3 productos vigentes en categoría dada de baja|OK
C4 pedidos vigentes de usuarios dados de baja|OK
C5 pedidos confirmados o terminados sin detalle|OK
```

Dos fallas reales detectadas por esta prueba durante la preparación del apunte

- La primera versión ejecutaba la restauración con el rol `admin_datos`. Falló con `permission denied to create extension "pg_cron"`: restaurar extensiones requiere un superusuario. Por esa razón, la prueba se ejecuta en el **servidor de pruebas** con su superusuario, nunca en el productivo.
- La segunda versión falló con `schema "cron" does not exist`: la extensión `pg_cron` (sección 4.10) solo puede existir en la base indicada en `cron.database_name`. La solución fue filtrar el índice del respaldo (línea 24 del script) y restaurar con `-L`.

Ninguno de los dos problemas se habría descubierto sin ensayar la restauración. Ambos se habrían descubierto durante un incidente real, con el servicio caído.

Cuando la prueba se ejecuta en un servidor de pruebas distinto del productivo, la comparación de conteos se hace contra valores que el script de respaldo registra en la bitácora en el momento de respaldar, ya que la base productiva sigue recibiendo pedidos después del respaldo.

3. Automatización del respaldo

Un respaldo que depende de que una persona recuerde ejecutarlo deja de hacerse en la primera semana con mucho trabajo. El respaldo se automatiza con un script que cumple requisitos explícitos y se programa en el planificador del sistema operativo. Se programa **fuera de la base** porque `pg_dump` es un programa externo al motor: si el servidor de base de datos está caído, el planificador del sistema operativo sigue funcionando y registra la falla.

3.1. Requisitos del script de respaldo

Antes de escribir (o de pedir a la IA que escriba) el script, se fijan los requisitos que debe cumplir. Esta lista es la especificación contra la que después se revisa el código, línea por línea:

N.º	Requisito	Por qué
R1	Se conecta con `mant_backup`, nunca con un superusuario	Mínimo privilegio (Unidad 5)
R2	No contiene contraseñas; las toma de `.pgpass`	El script se versiona en Git
R3	Se detiene ante el primer error	Un paso fallido no debe dar lugar a un respaldo «exitoso» vacío
R4	Escribe en un archivo temporal y lo renombra solo al final	Un respaldo truncado nunca tiene el nombre de un respaldo válido

N.º	Requisito	Por qué
R5	Verifica que el archivo se pueda leer con `pg_restore --list`	Detección inmediata de archivos corruptos
R6	Calcula una huella SHA-256 del archivo	Permite verificar que la copia externa no se alteró
R7	Borra solo los archivos de su propio patrón y de más de N días	La retención nunca puede borrar otra cosa
R8	Registra el resultado (OK o ERROR) en `mantenimiento.bitacora`	El monitoreo detecta respaldos faltantes

3.2. Script de respaldo para Linux

Archivo backup_food_store.sh

```
#!/usr/bin/env bash
# ============================================================
# Unidad 6 · backup_food_store.sh
# Respaldo lógico diario de Food Store con verificación y retención.
# La contraseña NO está en el script: se toma de ~/.pgpass (permiso 600).
# ============================================================
set -euo pipefail                       # cortar ante cualquier error o variable no definida

PGHOST="${PGHOST:-localhost}"
PGUSER="mant_backup"                    # rol de mínimo privilegio (solo lectura)
BASE="food_store"
DESTINO="${DESTINO:-/var/backups/food_store}"
RETENCION_DIAS="${RETENCION_DIAS:-14}"
FECHA="$(date +%Y%m%d_%H%M)"
ARCHIVO="${DESTINO}/${BASE}_${FECHA}.dump"

# Protección: nunca operar sobre un destino vacío o la raíz
[[ -n "$DESTINO" && "$DESTINO" != "/" ]] || { echo "DESTINO inválido"; exit 1; }
mkdir -p "$DESTINO"

INICIO="$(date '+%F %T%z')"
echo "$INICIO Inicio respaldo -> $ARCHIVO"

# Registro en la bitácora de la base (rol_respaldo solo tiene INSERT sobre ella)
registrar() {   # $1 = OK | ERROR   $2 = detalle
  psql -h "$PGHOST" -U "$PGUSER" -d "$BASE" -X -q -v ON_ERROR_STOP=1 \
       -v ini="$INICIO" -v est="$1" -v det="$2" <<'SQL'
INSERT INTO mantenimiento.bitacora(tarea, inicio, fin, estado, detalle)
VALUES ('respaldo_logico', :'ini', clock_timestamp(), :'est', :'det');
SQL
}
trap 'rm -f "${ARCHIVO}.parcial"; registrar ERROR "falló en la línea $LINENO"' ERR

# 1) Respaldo lógico en formato custom (comprimido, restauración selectiva).
#    Los datos de la agenda de pg_cron se excluyen: están protegidos con RLS
#    y su definición ya está versionada en 06_tareas_programadas.sql
pg_dump -h "$PGHOST" -U "$PGUSER" -d "$BASE" -Fc -Z 6 \
        --exclude-table-data='cron.*' -f "${ARCHIVO}.parcial"

# 2) Verificación mínima: el archivo debe poder leerse con pg_restore
pg_restore --list "${ARCHIVO}.parcial" > /dev/null

# 3) Recién ahora el respaldo se considera válido (renombrado atómico)
mv "${ARCHIVO}.parcial" "$ARCHIVO"
sha256sum "$ARCHIVO" > "${ARCHIVO}.sha256"
```

```
# 4) Retención: borrar SOLO archivos de este patrón y más antiguos que N días
find "$DESTINO" -maxdepth 1 -type f \
     \( -name "${BASE}_*.dump" -o -name "${BASE}_*.dump.sha256" \) \
     -mtime +"$RETENCION_DIAS" -print -delete

TAMANO="$(du -h "$ARCHIVO" | cut -f1)"
registrar OK "$(basename "$ARCHIVO") $TAMANO"
echo "$(date '+%F %T') Fin OK ($TAMANO)"
```

La tabla siguiente vincula cada decisión del script con el requisito que cumple. Este es el tipo de lectura línea por línea que se exige en la defensa de cualquier script de la cursada:

Líneas	Qué hace	Requisito
7	`set -euo pipefail`: termina ante un comando fallido, una variable no definida o una falla dentro de una tubería	R3
10	Usuario fijo `mant_backup`; la contraseña la resuelve libpq desde `~/.pgpass`	R1, R2
18	Rechaza un destino vacío o la raíz del sistema antes de crear o borrar nada	R7
25-32	Función `registrar` y `trap ... ERR`: ante cualquier error borra el archivo parcial y deja una fila `ERROR` en la bitácora	R4, R8
37-38	`pg_dump` en formato personalizado hacia un archivo `.parcial`	R4
41	`pg_restore --list` comprueba que el archivo sea legible	R5
44-45	Renombrado final y huella SHA-256	R4, R6
48-50	`find` limitado a la carpeta (`-maxdepth 1`), al patrón del nombre y a la antigüedad	R7
53	Fila `OK` en la bitácora con nombre y tamaño del archivo	R8

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
2026-10-05 21:14:41 Inicio respaldo -> /var/lib/pgdemo/backups/food_store_20261006_0014.dump
2026-10-05 21:14:43 Fin OK (12M)

SELECT id, tarea, fin::time(0) AS fin, estado, detalle, ejecutado_por
FROM mantenimiento.bitacora WHERE tarea = 'respaldo_logico';

 id |      tarea      |   fin    | estado |              detalle               | ejecutado_por
----+-----------------+----------+--------+------------------------------------+---------------
 34 | respaldo_logico | 21:14:44 | OK     | food_store_20261006_0014.dump 12M | mant_backup
```

Para que `mant_backup` pueda registrar su ejecución sin obtener ningún otro permiso de escritura, se le otorga únicamente `INSERT` sobre la bitácora (script `05_monitoreo_salud.sql`, sección 5.3). No puede leerla, modificarla ni borrarla: puede dejar constancia, pero no puede alterar la constancia que dejaron otros.

3.3. Equivalente para Windows

En los equipos del laboratorio con Windows, el mismo procedimiento se implementa en PowerShell y se programa con el Programador de tareas. Los requisitos son los mismos; cambia la sintaxis:

Archivo backup_food_store.ps1

```
# ============================================================
# Unidad 6 · backup_food_store.ps1  (equivalente para Windows)
# Respaldo lógico diario de Food Store con verificación y retención.
# La contraseña se toma de %APPDATA%\postgresql\pgpass.conf
# ============================================================
$ErrorActionPreference = 'Stop'

$PgBin     = 'C:\Program Files\PostgreSQL\16\bin'
$PgHost    = 'localhost'
$PgUser    = 'mant_backup'
$Base      = 'food_store'
$Destino   = 'D:\Respaldos\food_store'
$Retencion = 14
$Fecha     = Get-Date -Format 'yyyyMMdd_HHmm'
$Archivo   = Join-Path $Destino "$($Base)_$Fecha.dump"
$Parcial   = "$Archivo.parcial"

New-Item -ItemType Directory -Force -Path $Destino | Out-Null
Write-Output "$(Get-Date -Format s) Inicio respaldo -> $Archivo"

# 1) Respaldo en formato custom
& "$PgBin\pg_dump.exe" -h $PgHost -U $PgUser -d $Base -Fc -Z 6 -f $Parcial
if ($LASTEXITCODE -ne 0) { throw "pg_dump terminó con código $LASTEXITCODE" }

# 2) Verificación mínima: el archivo debe poder listarse
& "$PgBin\pg_restore.exe" --list $Parcial | Out-Null
if ($LASTEXITCODE -ne 0) { throw "El respaldo no es legible" }

# 3) Renombrado y huella digital
Move-Item $Parcial $Archivo
(Get-FileHash $Archivo -Algorithm SHA256).Hash | Set-Content "$Archivo.sha256"

# 4) Retención: solo archivos de este patrón y más antiguos que N días
Get-ChildItem $Destino -File -Filter "$($Base)_*.dump*" |
    Where-Object { $_.LastWriteTime -lt (Get-Date).AddDays(-$Retencion) } |
    Remove-Item -Verbose

Write-Output "$(Get-Date -Format s) Fin OK"
```

Diferencia que hay que conocer

En PowerShell, `$ErrorActionPreference = 'Stop'` detiene el script ante errores de los comandos de PowerShell, pero **no** ante un programa externo que termina con error, como `pg_dump.exe`. Por eso el script verifica explícitamente `$LASTEXITCODE` después de cada programa externo. Omitir esa verificación produce un script que informa «Fin OK» aunque el respaldo haya fallado.

3.4. Programación en el sistema operativo

El planificador del sistema operativo ejecuta el respaldo a la hora indicada aunque nadie haya iniciado sesión. En Linux se usa `cron`; en Windows, el Programador de tareas (`schtasks`).

Archivo agenda_sistema_operativo.txt

```
# ---------- Linux: crontab del usuario de sistema que ejecuta los respaldos ----------
# (editar con: crontab -e)   minuto hora día mes día_semana  comando
0  2 * * *  /opt/food_store/backup_food_store.sh   >> /var/log/food_store/backup.log 2>&1
0  6 * * 0  /opt/food_store/prueba_restauracion.sh >> /var/log/food_store/restore_test.log 2>&1
```

```
# ---------- Windows: Programador de tareas (consola con permisos de administrador) ----------
schtasks /Create /TN "FoodStore\Respaldo diario" /SC DAILY /ST 02:00 /RU "SVC_BACKUP" /RP * ^
         /TR "powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\food_store\backup_food_store.ps1"
schtasks /Query /TN "FoodStore\Respaldo diario" /V /FO LIST
```

Campo de cron	Valores	Ejemplo `0 2 * * *`
Minuto	0-59	0
Hora	0-23	2 (las 02:00)
Día del mes	1-31	* (todos)
Mes	1-12	* (todos)
Día de la semana	0-6 (0 = domingo)	* (todos)

3.5. Caso real: el respaldo que dejó de funcionar

Incidente registrado durante la preparación de este apunte

El script de respaldo funcionó correctamente durante las primeras ejecuciones. Después se instaló la extensión `pg_cron` (sección 4.10) y la ejecución siguiente falló con este mensaje:

```
pg_dump: error: query failed: ERROR: query would be affected by row-level security policy for table "job"
```

La extensión protege su tabla `cron.job` con seguridad a nivel de fila (el mecanismo RLS estudiado en la Unidad 5) y `pg_read_all_data` no alcanza para leerla. El cambio de una pieza ajena al script rompió el respaldo sin que nadie modificara el script.

La falla se detectó de inmediato porque el script cumplía los requisitos R3, R4 y R8: se detuvo, el archivo incompleto quedó con extensión `.parcial` (nunca con el nombre de un respaldo válido) y se registró la fila `ERROR` en la bitácora. La línea 35 correspondía al `pg_dump` en esa versión del script; a raíz de este incidente se agregó al `trap` el borrado del archivo parcial (línea 32 de la versión final):

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
 id |      tarea      | estado |              detalle               | ejecutado_por
----+-----------------+--------+------------------------------------+---------------
 34 | respaldo_logico | OK     | food_store_20261006_0014.dump 12M | mant_backup
 37 | respaldo_logico | ERROR  | falló en la línea 35               | mant_backup
```

La corrección consistió en excluir los datos de la agenda de `pg_cron` del respaldo lógico (`--exclude-table-data='cron.*'`, línea 38 del script), porque la definición de esas tareas ya está versionada en el script `06_tareas_programadas.sql`. La decisión se documenta en el plan de mantenimiento. Un script sin esas protecciones habría seguido generando, cada noche, un archivo vacío con nombre de respaldo válido.

4. Tareas programadas de mantenimiento

4.1. MVCC y tuplas muertas

La Unidad 1 explicó que PostgreSQL implementa el aislamiento entre transacciones con **MVCC** (Multi-Version Concurrency Control, control de concurrencia multiversión): las lecturas no bloquean a las escrituras ni las escrituras a las lecturas. El precio de ese mecanismo es que un `UPDATE` no sobrescribe la fila: crea una versión nueva y marca la versión anterior como obsoleta, registrando en ella el identificador de la transacción que la reemplazó (`xmax`). Un `DELETE` hace lo mismo, sin crear versión nueva. Las versiones obsoletas se denominan **tuplas muertas** y ocupan espacio en la tabla y en sus índices hasta que un proceso las elimina.



Figura 7. *Una fila de producto antes y después de un UPDATE, y después de VACUUM.*

La versión muerta no se puede eliminar inmediatamente: mientras exista alguna transacción abierta que comenzó antes del `UPDATE`, esa transacción todavía tiene derecho a ver el precio anterior. El proceso que elimina las versiones que ya nadie puede ver es **VACUUM**.

4.2. VACUUM y VACUUM FULL: ensayo sobre la tabla producto

El ensayo siguiente mide el efecto real de una actualización masiva. Para observar el fenómeno sin interferencias se desactiva temporalmente el autovacuum de la tabla; en producción el autovacuum **siempre** permanece activo.

Ensayo de crecimiento y limpieza

```
ALTER TABLE producto SET (autovacuum_enabled = false);   -- solo para el ensayo

-- Consulta de control, ejecutada después de cada paso
SELECT pg_size_pretty(pg_relation_size('producto')) AS tamano_tabla,
       n_live_tup AS vivas, n_dead_tup AS muertas
FROM   pg_stat_user_tables WHERE relname = 'producto';

UPDATE producto SET precio = round(precio * 1.10, 2);   -- aumento general del 10 %
VACUUM (VERBOSE) producto;
VACUUM FULL producto;

ALTER TABLE producto RESET (autovacuum_enabled);         -- se restituye
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
-- Inicial
 tamano_tabla | vivas | muertas
 6904 kB      | 50000 |       0
-- Después del UPDATE masivo
 13 MB        | 50000 |   50000
-- Después de VACUUM
 13 MB        | 50000 |       0
INFO:  finished vacuuming "food_store.public.producto": index scans: 1
pages: 0 removed, 1725 remain, 1725 scanned (100.00% of total)
tuples: 50000 removed, 50000 remain, 0 are dead but not yet removable
-- Después de un segundo UPDATE masivo y otro VACUUM
 13 MB        | 50000 |       0
-- Después de VACUUM FULL
 6904 kB      | 50000 |       0
```

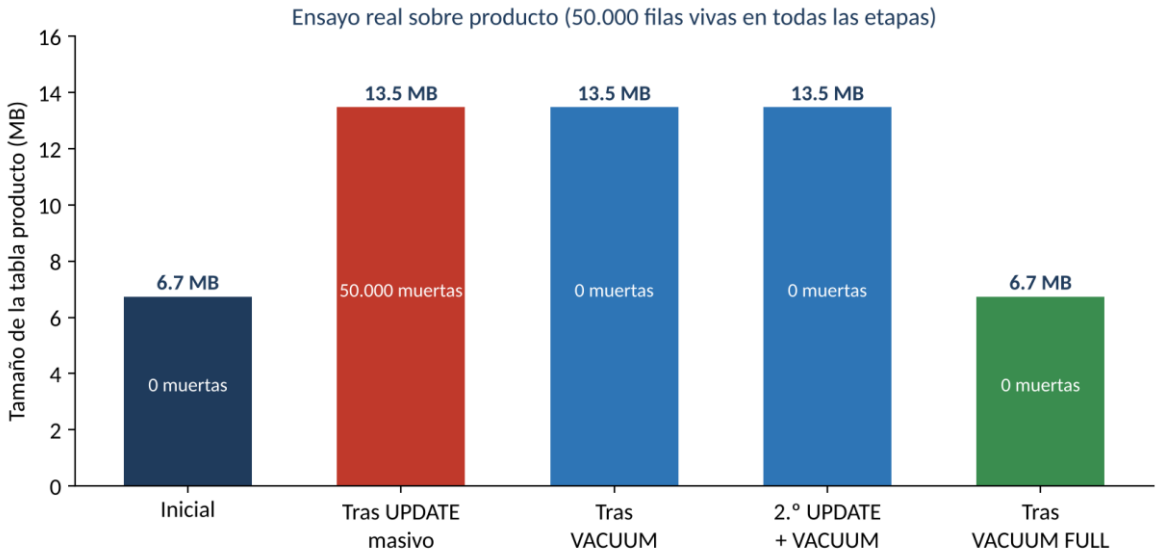


Figura 8. *Tamaño de la tabla producto en cada etapa del ensayo (datos medidos).*

El ensayo muestra cuatro hechos que conviene fijar:

1. Un `UPDATE` de todas las filas **duplica** el tamaño de la tabla, aunque la cantidad de filas vivas no cambia.
2. `VACUUM` elimina las 50.000 tuplas muertas pero **no reduce** el tamaño del archivo: marca el espacio como libre y reutilizable para futuras inserciones y actualizaciones.
3. El segundo `UPDATE` masivo **no hizo crecer** la tabla: reutilizó el espacio que liberó el `VACUUM` anterior. Esa es la situación estable de una tabla bien mantenida.
4. `VACUUM FULL` reescribe la tabla completa en un archivo nuevo y devuelve el espacio al sistema operativo, pero para hacerlo toma un bloqueo exclusivo (`ACCESS EXCLUSIVE`): mientras se ejecuta, **ninguna consulta puede leer ni escribir la tabla**.

Operación	Qué hace	Bloqueo	Cuándo usarla
`VACUUM`	Elimina tuplas muertas y marca el espacio como reutilizable; actualiza el mapa de visibilidad	No impide lecturas ni escrituras	En forma rutinaria (la ejecuta el autovacuum)

Operación	Qué hace	Bloqueo	Cuándo usarla
`VACUUM (ANALYZE)`	Lo anterior más la actualización de estadísticas	Igual que `VACUUM`	Tarea diaria programada sobre las tablas operativas
`VACUUM FULL`	Reescribe la tabla y sus índices; devuelve espacio al disco	Exclusivo: la tabla queda inaccesible	Excepcionalmente, en una ventana de mantenimiento, después de un borrado masivo

Error frecuente de alto impacto

Programar `VACUUM FULL` todas las noches «para que la base ocupe menos». En una tabla grande, cada ejecución deja la tabla inaccesible durante minutos; además, la tabla vuelve a crecer al día siguiente hasta su tamaño de equilibrio. Es una de las sugerencias incorrectas más frecuentes de los asistentes de IA cuando se les pide «un script de mantenimiento completo» (sección 6.3).

4.3. Autovacuum y su ajuste por tabla

El **autovacuum** es un proceso del motor, activo por omisión, que revisa periódicamente (cada `autovacuum_naptime`, 60 segundos) qué tablas superaron un umbral de cambios y ejecuta `VACUUM` o `ANALYZE` sobre ellas. El umbral para `VACUUM` se calcula así:

Fórmula del umbral de autovacuum (valores por omisión)

```
umbral_vacuum = autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor × filas_de_la_tabla
              =             50              +              0,20              × filas
```

Con los valores por omisión, la tabla `detalle_pedido` de Food Store (438.317 filas después del archivado de la sección 4.8) recién recibe un `VACUUM` automático cuando acumula **50 + 0,20 × 438.317 = 87.713** tuplas muertas. Para la tabla de mayor escritura del sistema, un 20 % de versiones muertas antes de limpiar es demasiado. El ajuste se hace **por tabla**, sin modificar el comportamiento global del servidor:

Ajuste de autovacuum para las tablas de mayor escritura

```
ALTER TABLE public.detalle_pedido SET (autovacuum_vacuum_scale_factor  = 0.05,
                                       autovacuum_analyze_scale_factor = 0.02);
ALTER TABLE public.pedido         SET (autovacuum_vacuum_scale_factor  = 0.05,
                                       autovacuum_analyze_scale_factor = 0.02);
```

Tabla detalle_pedido (438.317 filas)	Valores por omisión	Valores ajustados
Umbral para VACUUM automático	50 + 0,20 × 438.317 = 87.713 tuplas	50 + 0,05 × 438.317 = 21.966 tuplas
Umbral para ANALYZE automático	50 + 0,10 × 438.317 = 43.882 cambios	50 + 0,02 × 438.317 = 8.816 cambios

4.4. La transacción olvidada que impide limpiar

VACUUM solo elimina las versiones que ninguna transacción abierta puede ver. Una sesión que inicia una transacción y queda inactiva sin confirmarla (estado `idle in transaction`, típica de una herramienta gráfica abierta durante el almuerzo o de una aplicación con un error en el manejo de conexiones) impide la limpieza de **toda la base**, no solamente de las tablas que consultó. El ensayo siguiente lo demuestra:

Ensayo con una transacción olvidada

```
-- Sesión A (admin_datos): abre una transacción y queda inactiva
BEGIN ISOLATION LEVEL REPEATABLE READ;
SELECT count(*) FROM categoria;        -- no toca la tabla producto
-- ... la sesión queda abierta, sin COMMIT

-- Sesión B (administrador): actualización y limpieza de producto
UPDATE producto SET precio = precio;
VACUUM (VERBOSE) producto;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
UPDATE 50000
tuples: 0 removed, 100000 remain, 50000 are dead but not yet removable
removable cutoff: 930, which was 2 XIDs old when operation ended
```

El motor informa con precisión el problema: hay 50.000 tuplas muertas que **todavía no se pueden eliminar** (`dead but not yet removable`). La causa se encuentra en `pg_stat_activity`:

Búsqueda de sesiones inactivas con transacción abierta

```
SELECT pid, usename, state, now() - xact_start AS antiguedad_transaccion,
       left(query, 40) AS ultima_consulta
FROM   pg_stat_activity
WHERE  state = 'idle in transaction';
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
 pid |   usename   |        state        | antiguedad_transaccion |         ultima_consulta
-----+-------------+---------------------+------------------------+---------------------------------
 520 | admin_datos | idle in transaction | 00:00:02.322941        | SELECT count(*) FROM categoria;
```

Acción correctiva y verificación

```
SELECT pg_terminate_backend(520);   -- termina la sesión (su transacción se revierte)
VACUUM (VERBOSE) producto;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
 pg_terminate_backend
----------------------
 t
tuples: 50000 removed, 50000 remain, 0 are dead but not yet removable
```

La prevención es un parámetro del servidor: `idle_in_transaction_session_timeout` termina automáticamente las sesiones que permanecen inactivas con una transacción abierta durante más del tiempo indicado. En Food Store se fija en 10 minutos (script `07_config_monitoreo.sql`, sección

5.4). Terminar una sesión ajena es una decisión con consecuencias (la transacción de esa sesión se revierte), por lo que se registra en la bitácora quién lo hizo y por qué.

4.5. Congelamiento y agotamiento del contador de transacciones (nociones)

Cada transacción que modifica datos recibe un identificador (XID) de 32 bits. El motor compara XID para decidir qué versión de una fila es visible, y con 32 bits el contador da la vuelta después de unos cuatro mil millones de transacciones. Para que las filas antiguas no «parezcan del futuro» después de esa vuelta, `VACUUM` las **congela**: las marca como visibles para todas las transacciones. Si el congelamiento no se realiza a tiempo, el motor se protege a sí mismo dejando de aceptar escrituras. El autovacuum fuerza un congelamiento cuando la edad del XID más antiguo de una tabla supera `autovacuum_freeze_max_age` (200 millones por omisión), incluso si el autovacuum está desactivado para esa tabla. La métrica se controla en el tablero de la sección 5:

Edad del contador de transacciones

```
SELECT datname, age(datfrozenxid) AS edad_xid,
       current_setting('autovacuum_freeze_max_age') AS freeze_max_age
FROM   pg_database WHERE datname = 'food_store';
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
  datname   | edad_xid | freeze_max_age
------------+----------+----------------
 food_store |      608 |  200000000
```

4.6. ANALYZE y las estadísticas del optimizador

La Unidad 2 mostró que el optimizador elige el plan de ejecución a partir de **estadísticas**: cantidad de filas, valores más frecuentes e histogramas de cada columna. Esas estadísticas se calculan con `ANALYZE` (también lo ejecuta el autovacuum). Entre una carga masiva y el siguiente `ANALYZE`, el optimizador trabaja con una imagen desactualizada de los datos. El ensayo siguiente, ejecutado dentro de una transacción que se revierte al final, inserta 30.000 productos de promoción a $ 99,99 y compara la estimación del optimizador antes y después de `ANALYZE`:

Estimación de filas con estadísticas viejas y nuevas

```
BEGIN;
INSERT INTO producto (nombre, precio, stock, categoria_id)
SELECT 'Promo ' || i, 99.99, 100, 3 FROM generate_series(1, 30000) s(i);

EXPLAIN (ANALYZE, TIMING OFF, SUMMARY OFF)
SELECT * FROM producto WHERE precio < 100;

ANALYZE producto;

EXPLAIN (ANALYZE, TIMING OFF, SUMMARY OFF)
SELECT * FROM producto WHERE precio < 100;
ROLLBACK;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
-- Antes de ANALYZE
```

```
 Seq Scan on producto  (cost=0.00..2125.96 rows=7 width=605) (actual rows=30000 loops=1)
   Filter: (precio < '100'::numeric)
-- Después de ANALYZE
 Seq Scan on producto  (cost=0.00..2233.00 rows=30170 width=604) (actual rows=30000 loops=1)
   Filter: (precio < '100'::numeric)
```

Con las estadísticas viejas, el optimizador estimaba **7 filas**; el resultado real fue de **30.000**. Un error de estimación de esa magnitud es el origen típico de planes malos en consultas con reuniones: el optimizador elige un ciclo anidado pensando que procesa 7 filas y termina procesando 30.000. La regla operativa es ejecutar `ANALYZE` sobre las tablas afectadas **inmediatamente después de toda carga masiva**, como ya lo hacía la última línea del script `Genera_registros.sql` del proyecto.

4.7. Mantenimiento de índices

Reconstrucción sin bloquear la operación

Los índices B-tree también acumulan páginas con espacio desaprovechado después de muchas actualizaciones. `REINDEX` reconstruye un índice desde cero. La variante `CONCURRENTLY` (PostgreSQL 12+) construye el índice nuevo en paralelo y lo reemplaza al final, sin impedir lecturas ni escrituras sobre la tabla:

Reconstrucción de un índice sin bloquear la tabla

```
REINDEX (VERBOSE) INDEX CONCURRENTLY detalle_pedido_pkey;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
INFO:  index "public.detalle_pedido_pkey" was reindexed
DETAIL:  CPU: user: 0.09 s, system: 0.04 s, elapsed: 0.17 s.
```

Índices que nadie usa

La Unidad 3 advirtió que cada índice tiene un costo en cada escritura. La vista `pg_stat_user_indexes` registra cuántas veces se usó cada índice desde el último reinicio de estadísticas:

Uso de los índices

```
SELECT relname AS tabla, indexrelname AS indice, idx_scan AS usos,
       pg_size_pretty(pg_relation_size(indexrelid)) AS tamano
FROM   pg_stat_user_indexes
WHERE  schemaname = 'public'
ORDER  BY idx_scan, pg_relation_size(indexrelid) DESC;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
     tabla      |        indice        | usos | tamano
----------------+----------------------+------+---------
 detalle_pedido | detalle_pedido_pkey  |    0 | 11 MB
 usuario        | usuario_mail_key     |    0 | 1384 kB
 categoria      | categoria_nombre_key |    0 | 16 kB
 categoria      | categoria_pkey       |    2 | 16 kB
 usuario        | usuario_pkey         |    6 | 456 kB
 pedido         | pedido_pkey          |    9 | 8792 kB
```

Lectura correcta de esta salida

Un índice con cero usos **no** es automáticamente un índice para eliminar. Los tres primeros de la lista respaldan restricciones `PRIMARY KEY` y `UNIQUE`: el motor los utiliza para impedir duplicados en cada inserción, aunque ninguna consulta los recorra. Además, el contador se reinicia con cada reinicio de estadísticas y en una base de prueba refleja pocos minutos de actividad. Solo son candidatos a eliminar los índices **creados por el equipo** (no los de restricciones) con cero usos durante un período representativo de la carga real, por ejemplo un mes completo de operación.

4.8. Limpieza planificada: archivado de pedidos

La política de Food Store, definida desde la Unidad 1, es la **baja lógica**: nada se borra físicamente, se marca con `eliminado = TRUE`. Con el tiempo, la tabla `pedido` acumula pedidos cancelados y dados de baja que ninguna pantalla de la aplicación consulta, pero que cada recorrido secuencial lee y cada respaldo copia. El archivado los traslada a tablas históricas del esquema `mantenimiento`, donde siguen disponibles para auditoría. El procedimiento cumple las mismas reglas de seguridad que el resto de la cursada:

- Por omisión funciona en **modo simulación** (`p_simular = TRUE`): informa cuántas filas movería, sin modificar nada.
- Rechaza parámetros contrarios a la política (pedidos de menos de 3 meses).
- **Copia antes de borrar**, y borra primero el detalle y después el pedido (la clave foránea es `ON DELETE RESTRICT`).
- Todo el movimiento ocurre en un único bloque: si cualquier paso falla, se revierte completo y solo queda la fila `ERROR` en la bitácora.

Archivo 03_archivado_pedidos.sql

```
-- ============================================================
-- Unidad 6 · 03_archivado_pedidos.sql
-- Limpieza planificada: mover pedidos CANCELADOS o dados de baja,
-- más antiguos que p_meses, a las tablas históricas.
-- p_simular = TRUE (valor por omisión) solo informa, no modifica.
-- ============================================================
CREATE OR REPLACE PROCEDURE mantenimiento.sp_archivar_pedidos(
    p_meses   INTEGER DEFAULT 6,
    p_simular BOOLEAN DEFAULT TRUE)
LANGUAGE plpgsql AS $$
DECLARE
    v_log_id  BIGINT;
    v_corte   DATE := CURRENT_DATE - make_interval(months => p_meses);
    v_pedidos BIGINT;
    v_detalle BIGINT;
BEGIN
    IF p_meses < 3 THEN
        RAISE EXCEPTION 'Por política, no se archivan pedidos de menos de 3 meses (recibido: %)', p_meses;
    END IF;


    -- 1) Se registra el inicio y se confirma: la bitácora sobrevive aunque la tarea falle
    INSERT INTO mantenimiento.bitacora(tarea, detalle)
    VALUES ('archivar_pedidos', format('corte=%s simular=%s', v_corte, p_simular))
    RETURNING id INTO v_log_id;
    COMMIT;


    BEGIN
        -- 2) Conjunto candidato, calculado una sola vez
        CREATE TEMP TABLE tmp_candidatos ON COMMIT DROP AS
        SELECT id FROM public.pedido
        WHERE  fecha < v_corte
          AND (estado = 'CANCELADO' OR eliminado = TRUE);
```

```
        SELECT count(*) INTO v_pedidos FROM tmp_candidatos;
        SELECT count(*) INTO v_detalle
        FROM public.detalle_pedido d JOIN tmp_candidatos c ON c.id = d.pedido_id;


        IF NOT p_simular THEN
            -- 3) Copiar primero, borrar después (el detalle antes que el pedido: FK RESTRICT)
            INSERT INTO mantenimiento.pedido_historico
            SELECT p.*, now() FROM public.pedido p JOIN tmp_candidatos c USING (id);
            INSERT INTO mantenimiento.detalle_pedido_historico
            SELECT d.*, now() FROM public.detalle_pedido d
            JOIN tmp_candidatos c ON c.id = d.pedido_id;


            DELETE FROM public.detalle_pedido d USING tmp_candidatos c WHERE d.pedido_id = c.id;
            DELETE FROM public.pedido p        USING tmp_candidatos c WHERE p.id = c.id;
        END IF;


        UPDATE mantenimiento.bitacora
        SET fin = clock_timestamp(), estado = 'OK', filas = v_pedidos,
            detalle = detalle || format(' | pedidos=%s detalle=%s', v_pedidos, v_detalle)
        WHERE id = v_log_id;
    EXCEPTION WHEN OTHERS THEN
        -- Todo lo hecho en el bloque se revierte; solo queda constancia del error
        UPDATE mantenimiento.bitacora
        SET fin = clock_timestamp(), estado = 'ERROR', detalle = detalle || ' | ' || SQLERRM
        WHERE id = v_log_id;
    END;
    COMMIT;
    RAISE NOTICE 'Pedidos: %, líneas de detalle: % (simulación: %)', v_pedidos, v_detalle, p_simular;
END $$;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
CALL mantenimiento.sp_archivar_pedidos(2);
ERROR:  Por política, no se archivan pedidos de menos de 3 meses (recibido: 2)

CALL mantenimiento.sp_archivar_pedidos(6);          -- simulación
NOTICE:  Pedidos: 24539, líneas de detalle: 61393 (simulación: t)

CALL mantenimiento.sp_archivar_pedidos(6, FALSE);   -- ejecución real
NOTICE:  Pedidos: 24539, líneas de detalle: 61393 (simulación: f)

 pedidos_antes | pedidos_despues | pedidos_historicos | lineas_historicas
---------------+-----------------+--------------------+-------------------
        200001 |          175462 |              24539 |             61393

-- Efecto colateral: tuplas muertas generadas por el DELETE
    relname     | n_dead_tup
----------------+------------
 pedido         |      24542
 detalle_pedido |      61394
```

La última consulta conecta esta sección con la 4.2: el archivado borra físicamente 85.000 filas de las tablas operativas, que quedan como tuplas muertas hasta el siguiente `VACUUM`. Por eso, en la agenda de la sección 4.10, el archivado mensual (03:00) se programa **antes** del `VACUUM (ANALYZE)` diario (03:30), que limpia esas tuplas muertas y actualiza las estadísticas de las tablas que cambiaron.

Decisión de diseño documentada

El borrado físico de pedidos es una excepción a la política de baja lógica. La Unidad 5 definió que todo `DELETE` sobre las tablas del dominio queda auditado y que un `DELETE` fuera de una migración planificada es un incidente. El archivado **es** una operación planificada: está especificada, tiene modo de simulación, copia antes de borrar, se ejecuta con agenda conocida

y deja registro en la bitácora. Esa documentación es lo que distingue una limpieza planificada de una pérdida de datos.

4.9. Verificación de integridad

Integridad de negocio

Las restricciones declarativas (`CHECK`, `FOREIGN KEY`, `UNIQUE`) y los disparadores de la Unidad 3 protegen la mayoría de las reglas del dominio. Algunas reglas, sin embargo, involucran varias filas o varias tablas y pueden quedar violadas por un error de la aplicación, una corrección manual o un script defectuoso. Para ellas se escribe una función de control que devuelve, para cada regla, la cantidad de filas que la incumplen:

Archivo 04_verificar_integridad.sql

```
-- ============================================================
-- Unidad 6 · 04_verificar_integridad.sql
-- Controles de integridad de NEGOCIO que el motor no garantiza solo
-- ============================================================
CREATE OR REPLACE FUNCTION mantenimiento.fn_verificar_integridad()
RETURNS TABLE(control TEXT, filas_con_problema BIGINT, resultado TEXT)
LANGUAGE sql STABLE AS $$
    WITH c AS (
        SELECT 'C1 total del pedido = suma de subtotales' AS control,
               count(*) AS n
        FROM public.pedido p
        WHERE p.total <> public.calcular_total_pedido(p.id)
    UNION ALL
        SELECT 'C2 subtotal = cantidad x precio_unitario', count(*)
        FROM public.detalle_pedido
        WHERE subtotal <> cantidad * precio_unitario
    UNION ALL
        SELECT 'C3 productos vigentes en categoría dada de baja', count(*)
        FROM public.producto p JOIN public.categoria c ON c.id = p.categoria_id
        WHERE p.eliminado = FALSE AND c.eliminado = TRUE
    UNION ALL
        SELECT 'C4 pedidos vigentes de usuarios dados de baja', count(*)
        FROM public.pedido p JOIN public.usuario u ON u.id = p.usuario_id
        WHERE p.eliminado = FALSE AND u.eliminado = TRUE
    UNION ALL
        SELECT 'C5 pedidos confirmados o terminados sin detalle', count(*)
        FROM public.pedido p
        WHERE p.estado IN ('CONFIRMADO','TERMINADO')
          AND NOT EXISTS (SELECT 1 FROM public.detalle_pedido d WHERE d.pedido_id = p.id)
    )
    SELECT control, n, CASE WHEN n = 0 THEN 'OK' ELSE 'REVISAR' END FROM c;
$$;
```

Para comprobar que la función detecta lo que debe detectar, se inyectan dos defectos controlados: dos pedidos con el total alterado y un pedido terminado al que se le borró su detalle.

Prueba de la función con defectos inyectados

```
UPDATE pedido SET total = total + 100 WHERE id IN (1500, 1501);
DELETE FROM detalle_pedido
WHERE  pedido_id = (SELECT min(id) FROM pedido WHERE estado = 'TERMINADO');

SELECT * FROM mantenimiento.fn_verificar_integridad() WHERE resultado <> 'OK';
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
                     control                      | filas_con_problema | resultado
--------------------------------------------------+--------------------+-----------
 C1 total del pedido = suma de subtotales         |                  3 | REVISAR
 C5 pedidos confirmados o terminados sin detalle |                  1 | REVISAR

-- Detalle del control C1
  id  | total_registrado | total_calculado
------+------------------+-----------------
 1500 |          4724.32 |         4624.32
 1501 |         14462.04 |        14362.04
    2 |          6282.04 |               0
```

La función encontró los dos totales alterados y, además, un tercer caso que no se buscaba: al borrar el detalle del pedido 2, su total quedó desactualizado, porque el disparador `trg_total_*` de la Unidad 3 recalcula el total ante `INSERT` y `UPDATE` de líneas, **pero no ante `DELETE`**. El control de integridad descubrió un hueco de diseño en un objeto de una unidad anterior. Ese hallazgo se registra y se corrige en el repositorio (agregando el disparador `AFTER DELETE` correspondiente), no solamente en los datos.

Integridad física: amcheck y sumas de verificación

La extensión `amcheck`, incluida en la distribución estándar de PostgreSQL, verifica la estructura interna de los índices B-tree: que las claves estén ordenadas, que los punteros entre páginas sean coherentes y, con el segundo parámetro en `true`, que cada fila de la tabla tenga su entrada en el índice. Un índice dañado puede devolver resultados incorrectos sin producir ningún error visible.

Verificación estructural de los índices de Food Store

```
CREATE EXTENSION IF NOT EXISTS amcheck;

SELECT c.relname AS indice, 'OK' AS estructura
FROM   pg_index i
JOIN   pg_class c      ON c.oid = i.indexrelid
JOIN   pg_namespace n  ON n.oid = c.relnamespace
JOIN   pg_am a         ON a.oid = c.relam
CROSS  JOIN LATERAL (SELECT bt_index_check(c.oid, true)) AS chk   -- lanza error si hay daño
WHERE  n.nspname = 'public' AND a.amname = 'btree'
ORDER  BY 1;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
                 indice                 | estructura
----------------------------------------+------------
 categoria_nombre_key                   | OK
 categoria_pkey                         | OK
 detalle_pedido_pedido_id_producto_id_key | OK
 detalle_pedido_pkey                    | OK
 pedido_pkey                            | OK
 producto_pkey                          | OK
 usuario_mail_key                       | OK
 usuario_pkey                           | OK
Time: 338.528 ms
```

En el nivel de los archivos de datos, PostgreSQL puede calcular una **suma de verificación** (checksum) por cada página de 8 kB y comprobarla en cada lectura, lo que detecta corrupción producida por el disco o el controlador. Se habilita al crear el clúster (`initdb --data-checksums`) o, con el servidor

detenido, con la herramienta `pg_checksums --enable`. Su estado se consulta con `SHOW data_checksums;`. Desde PostgreSQL 18, los clústeres nuevos se crean con las sumas de verificación habilitadas por omisión.

4.10. Programación de las tareas

Qué se programa dónde

Tarea	Dónde se programa	Motivo
Respaldo lógico, prueba de restauración	Sistema operativo (cron / Programador de tareas)	Son programas externos al motor y deben ejecutarse aunque el motor falle
VACUUM, ANALYZE, controles de integridad, amcheck, archivado	Dentro del motor, con `pg_cron`	Son sentencias SQL; la agenda queda junto a la base y se consulta con SQL
Archivado de WAL	El propio motor (`archive_command`)	Se ejecuta cada vez que se completa un segmento

pg_cron: el planificador dentro de la base

`pg_cron` es una extensión de código abierto (licencia PostgreSQL) que agrega al motor un planificador con la misma sintaxis de cinco campos que `cron`. Cada tarea se ejecuta con los permisos del rol que la programó; por eso las tareas de Food Store se programan conectado como `admin_datos`, el dueño de las tablas, que en PostgreSQL 16 es el único rol no superusuario habilitado para ejecutar `VACUUM` sobre ellas.

Archivo 06_tareas_programadas.sql

```
-- ============================================================
-- Unidad 6 · 06_tareas_programadas.sql
-- Ajuste de autovacuum por tabla y agenda de tareas con pg_cron.
-- Se ejecuta conectado como admin_datos (dueño de las tablas).
-- Requisito previo (superusuario, una sola vez):
--   postgresql.conf: shared_preload_libraries = 'pg_cron'
--                    cron.database_name = 'food_store'
--   CREATE EXTENSION pg_cron;  GRANT USAGE ON SCHEMA cron TO admin_datos;
-- ============================================================

-- 1) Autovacuum más agresivo en las tablas de mayor escritura.
--    Por omisión actúa cuando cambia el 20 % de la tabla; aquí, el 5 %.
ALTER TABLE public.detalle_pedido SET (autovacuum_vacuum_scale_factor  = 0.05,
                                       autovacuum_analyze_scale_factor = 0.02);
ALTER TABLE public.pedido         SET (autovacuum_vacuum_scale_factor  = 0.05,
                                       autovacuum_analyze_scale_factor = 0.02);

-- 2) Agenda (formato cron: minuto hora día-del-mes mes día-de-la-semana)

-- Día 1 de cada mes 03:00 — archivado de pedidos cancelados de más de 12 meses
-- (antes del VACUUM de las 03:30, que limpia las tuplas muertas que deja el borrado)
SELECT cron.schedule('fs_archivado_mensual', '0 3 1 * *',
  $$CALL mantenimiento.sp_archivar_pedidos(12, FALSE)$$);

-- Diario 03:30 — VACUUM + estadísticas de las tablas operativas
SELECT cron.schedule('fs_vacuum_analyze_diario', '30 3 * * *',
  $$VACUUM (ANALYZE) public.pedido, public.detalle_pedido, public.producto$$);

-- Diario 04:00 — controles de integridad de negocio, resultado a la bitácora
SELECT cron.schedule('fs_integridad_diaria', '0 4 * * *', $$
```

```
  INSERT INTO mantenimiento.bitacora(tarea, fin, estado, filas, detalle)
  SELECT 'verificar_integridad', clock_timestamp(),
         CASE WHEN sum(filas_con_problema) = 0 THEN 'OK' ELSE 'ERROR' END,
         sum(filas_con_problema),
         string_agg(left(control, 2) || '=' || filas_con_problema, ' ')
  FROM mantenimiento.fn_verificar_integridad() $$);

-- Domingo 04:30 — verificación estructural de los índices B-tree (amcheck)
SELECT cron.schedule('fs_amcheck_semanal', '30 4 * * 0', $$
  SELECT bt_index_check(c.oid, true)
  FROM pg_index i JOIN pg_class c ON c.oid = i.indexrelid
  JOIN pg_namespace n ON n.oid = c.relnamespace JOIN pg_am a ON a.oid = c.relam
  WHERE n.nspname = 'public' AND a.amname = 'btree' $$);

-- 3) Consultar la agenda y el resultado de las últimas ejecuciones
SELECT jobid, jobname, schedule, username, active FROM cron.job ORDER BY jobid;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
 jobid |         jobname          |  schedule   |  username   | active
-------+--------------------------+-------------+-------------+--------
     1 | fs_archivado_mensual     | 0 3 1 * *   | admin_datos | t
     2 | fs_vacuum_analyze_diario | 30 3 * * *  | admin_datos | t
     3 | fs_integridad_diaria     | 0 4 * * *   | admin_datos | t
     4 | fs_amcheck_semanal       | 30 4 * * 0  | admin_datos | t
```

Para comprobar que la agenda funciona sin esperar hasta las 04:00, se programó una copia temporal del control de integridad para que se ejecute cada minuto, se esperaron dos ejecuciones y se consultó el historial que mantiene `pg_cron`:

Prueba de la agenda

```
SELECT cron.schedule('fs_prueba_integridad', '* * * * *',
       (SELECT command FROM cron.job WHERE jobname = 'fs_integridad_diaria'));
-- ... dos minutos después:
SELECT j.jobname, d.status, d.return_message, d.start_time, d.end_time - d.start_time AS duracion
FROM   cron.job_run_details d JOIN cron.job j USING (jobid)
ORDER  BY d.start_time DESC LIMIT 3;
SELECT cron.unschedule('fs_prueba_integridad');   -- se elimina la tarea de prueba
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
       jobname        |  status   | return_message |       inicio        |    duracion
----------------------+-----------+----------------+---------------------+-----------------
 fs_prueba_integridad | succeeded | INSERT 0 1     | 2026-10-05 21:17:00 | 00:00:01.69774
 fs_prueba_integridad | succeeded | INSERT 0 1     | 2026-10-05 21:16:00 | 00:00:01.792285

        tarea         | estado | filas |         detalle
----------------------+--------+-------+--------------------------
 verificar_integridad | OK     |     0 | C1=0 C2=0 C3=0 C4=0 C5=0
```

Requisitos de instalación de pg_cron

- Se carga al iniciar el servidor: `shared_preload_libraries = 'pg_cron'` y reinicio del servicio.
- Solo puede existir en **una** base del clúster, la indicada en `cron.database_name` (por omisión, `postgres`). Esto tiene consecuencias en los respaldos y en las restauraciones (secciones 2.8 y 3.5).

- Por omisión, cada tarea abre una conexión al servidor: el rol que la programó debe poder autenticarse (archivo `.pgpass` del usuario del servicio o regla de `pg_hba.conf`). La alternativa es `cron.use_background_workers = on`.
- En Windows no hay paquetes oficiales; en ese caso, las tareas SQL se programan con el Programador de tareas invocando `psql -f`.

La agenda de Food Store

Las tareas se distribuyen en la franja de menor actividad del sistema y en un orden que respeta sus dependencias: primero el respaldo (antes de cualquier modificación), después el archivado mensual, la limpieza y las estadísticas, y al final los controles. La prueba de restauración se ejecuta el domingo, sobre el respaldo de esa madrugada.

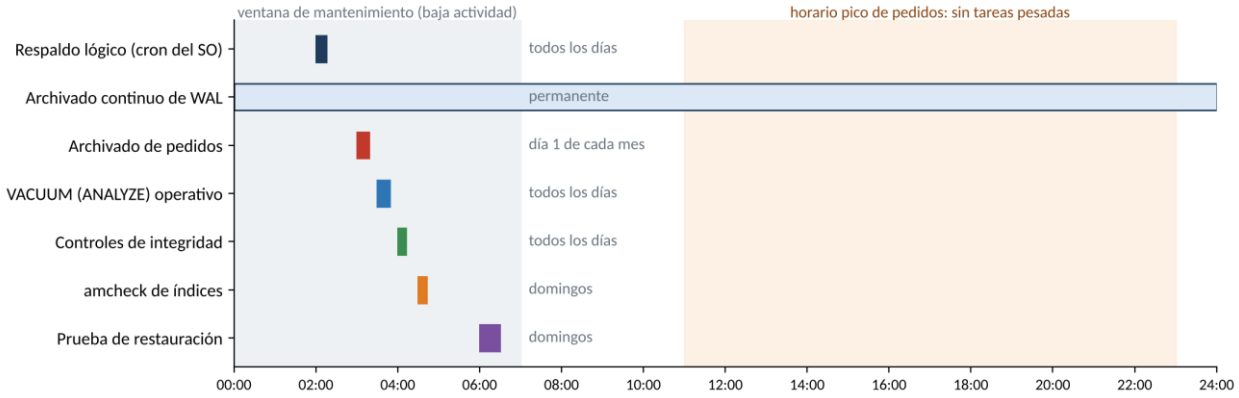


Figura 9. *Agenda de mantenimiento de Food Store en un día tipo.*

5. Monitoreo de la salud de la base

5.1. Qué medir y por qué

Monitorear no es mirar el servidor cuando alguien se queja. Es medir en forma periódica un conjunto acotado de indicadores, compararlos contra umbrales definidos de antemano y actuar cuando un indicador cruza su umbral, **antes** de que el problema llegue al usuario. Cada métrica responde a una pregunta concreta sobre el estado del motor:

Métrica	Pregunta que responde	Fuente en PostgreSQL
Conexiones en uso	¿Se está agotando el máximo de conexiones?	`pg_stat_activity`, `max_connections`
Aciertos de caché	¿Los datos más usados caben en memoria?	`pg_stat_database` (blks_hit, blks_read)
Transacción más antigua	¿Hay una transacción que impide limpiar (sección 4.4)?	`pg_stat_activity.xact_start`
Sesiones `idle in transaction`	¿Hay conexiones olvidadas con transacción abierta?	`pg_stat_activity.state`

Métrica	Pregunta que responde	Fuente en PostgreSQL
Sesiones bloqueadas	¿Hay sesiones esperando un bloqueo de otra (Unidad 1)?	`pg_blocking_pids()`
Tuplas muertas	¿El autovacuum está al día?	`pg_stat_user_tables`
Edad del XID	¿Hay riesgo de agotamiento del contador (sección 4.5)?	`pg_database.datfrozenxid`
Horas desde el último respaldo	¿El respaldo programado se está ejecutando con éxito?	`mantenimiento.bitacora`
Consultas más costosas	¿Qué consultas consumen más tiempo total (Unidad 2)?	`pg_stat_statements`
Tamaño de la base y de las tablas	¿Cuánto crece y cuándo se va a llenar el disco?	`pg_database_size()`, `pg_total_relation_size()`

5.2. Las vistas de estadísticas del motor

PostgreSQL mantiene un sistema de estadísticas acumuladas que se consulta con SQL. Las vistas principales para el administrador son `pg_stat_activity` (una fila por conexión, con su estado y su consulta actual), `pg_stat_database` (contadores por base), `pg_stat_user_tables` y `pg_stat_user_indexes` (actividad por tabla e índice) y, con la extensión `pg_stat_statements`, estadísticas agregadas por tipo de consulta. Las consultas de las secciones 4.4 y 4.7 ya usaron varias de ellas. La consulta siguiente identifica las sentencias que más tiempo total consumieron en Food Store:

Consultas que más tiempo total consumen

```
-- Requiere shared_preload_libraries = 'pg_stat_statements' y:
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

SELECT calls AS ejecuciones,
       round(total_exec_time::numeric, 1) AS ms_total,
       round(mean_exec_time::numeric, 1)  AS ms_promedio,
       left(regexp_replace(query, '\s+', ' ', 'g'), 70) AS consulta
FROM   pg_stat_statements
WHERE  dbid = (SELECT oid FROM pg_database WHERE datname = 'food_store')
ORDER  BY total_exec_time DESC
LIMIT  4;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
 ejecuciones | ms_total | ms_promedio |                                  consulta
-------------+----------+-------------+-----------------------------------------------------------------------------
           3 |    121.6 |        40.5 | SELECT u.mail, count(*) FROM pedido p JOIN usuario u ON u.id=p.usuario
           3 |     12.0 |         4.0 | SELECT * FROM v_pedidos_resumen WHERE usuario LIKE $1 LIMIT $2
           1 |      0.7 |         0.7 | SELECT pg_stat_statements_reset()
           3 |      0.2 |         0.1 | SELECT * FROM producto WHERE id = $1
```

Obsérvese que `pg_stat_statements` reemplaza los valores literales por parámetros (`$1`, `$2`): agrupa todas las ejecuciones de una misma forma de consulta, sin registrar los datos que se buscaron. Esta característica la vuelve apta para compartir con un asistente de IA sin exponer datos personales (sección 6.4).

5.3. Un tablero de salud con semáforo

Revisar diez consultas por separado no es práctico. La vista siguiente reúne las métricas principales en una fila por indicador, cada una con su umbral de alerta, su umbral crítico y un estado de semáforo. Los umbrales son valores iniciales propuestos por la cátedra: se ajustan cuando se conoce el comportamiento real de la base durante algunas semanas.

Archivo 05_monitoreo_salud.sql

```
-- ============================================================
-- Unidad 6 · 05_monitoreo_salud.sql
-- Tablero de salud de Food Store: una fila por métrica, con semáforo
-- Umbrales iniciales de cátedra: se ajustan con la historia real de la base
-- ============================================================
CREATE OR REPLACE VIEW mantenimiento.v_salud AS
WITH m AS (
    -- M1 Conexiones en uso respecto del máximo configurado (%)
    SELECT 1 AS orden, 'Conexiones en uso (%)' AS metrica,
           round(100.0 * count(*) / current_setting('max_connections')::int, 1) AS valor,
           70 AS umbral_alerta, 90 AS umbral_critico, 'mayor_es_peor' AS sentido
    FROM pg_stat_activity
  UNION ALL
    -- M2 Proporción de lecturas resueltas en memoria (cache hit ratio, %)
    SELECT 2, 'Aciertos de caché (%)',
           round(100.0 * blks_hit / NULLIF(blks_hit + blks_read, 0), 2),
           99, 95, 'menor_es_peor'
    FROM pg_stat_database WHERE datname = current_database()
  UNION ALL
    -- M3 Transacción abierta más antigua (minutos)
    SELECT 3, 'Transacción más antigua (min)',
           COALESCE(round(extract(epoch FROM max(now() - xact_start)) / 60, 1), 0),
           5, 30, 'mayor_es_peor'
    FROM pg_stat_activity
    WHERE xact_start IS NOT NULL AND pid <> pg_backend_pid()
  UNION ALL
    -- M4 Sesiones "idle in transaction" de más de 5 minutos
    SELECT 4, 'Sesiones inactivas con transacción abierta',
           count(*), 1, 3, 'mayor_es_peor'
    FROM pg_stat_activity
    WHERE state = 'idle in transaction' AND now() - state_change > interval '5 minutes'
  UNION ALL
    -- M5 Sesiones bloqueadas esperando a otra
    SELECT 5, 'Sesiones bloqueadas', count(*), 1, 5, 'mayor_es_peor'
    FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid)) > 0
  UNION ALL
    -- M6 Peor proporción de tuplas muertas entre las tablas del dominio (%)
    SELECT 6, 'Tuplas muertas, peor tabla (%)',
           COALESCE(max(round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1)), 0),
           20, 40, 'mayor_es_peor'
    FROM pg_stat_user_tables WHERE schemaname = 'public'
  UNION ALL
    -- M7 Consumo del contador de transacciones (wraparound) respecto del límite de congelado (%)
    SELECT 7, 'Edad de XID respecto de freeze_max_age (%)',
           round(100.0 * age(datfrozenxid) / current_setting('autovacuum_freeze_max_age')::bigint, 1),
           50, 80, 'mayor_es_peor'
    FROM pg_database WHERE datname = current_database()
  UNION ALL
    -- M8 Horas desde el último respaldo exitoso registrado en la bitácora
    SELECT 8, 'Horas desde el último respaldo OK',
           COALESCE(round(extract(epoch FROM now() - max(fin)) / 3600, 1), 9999),
           26, 48, 'mayor_es_peor'
    FROM mantenimiento.bitacora WHERE tarea = 'respaldo_logico' AND estado = 'OK'
)
SELECT orden, metrica, valor, umbral_alerta, umbral_critico,
       CASE
         WHEN sentido = 'mayor_es_peor' AND valor >= umbral_critico THEN 'ROJO'
         WHEN sentido = 'mayor_es_peor' AND valor >= umbral_alerta  THEN 'AMARILLO'
         WHEN sentido = 'menor_es_peor' AND valor <  umbral_critico THEN 'ROJO'
         WHEN sentido = 'menor_es_peor' AND valor <  umbral_alerta  THEN 'AMARILLO'
         ELSE 'VERDE'
       END AS semaforo
FROM m ORDER BY orden;


GRANT USAGE  ON SCHEMA mantenimiento TO rol_monitoreo, rol_respaldo;
GRANT SELECT ON mantenimiento.v_salud TO rol_monitoreo;
-- El script de respaldo deja constancia en la bitácora: solo INSERT, nada más
GRANT INSERT ON mantenimiento.bitacora TO rol_respaldo;
```

La vista se consulta con el rol de mínimo privilegio `mant_monitor`, que puede leer las métricas pero no los datos de negocio. La primera consulta del tablero en el laboratorio de la cátedra produjo este resultado:

Consulta del tablero de salud

```
-- Conectado como mant_monitor
SELECT metrica, valor, umbral_alerta AS alerta, umbral_critico AS critico, semaforo
FROM   mantenimiento.v_salud;
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
                  metrica                   | valor | alerta | critico | semaforo
--------------------------------------------+-------+--------+---------+----------
 Conexiones en uso (%)                      |   7.0 |     70 |      90 | VERDE
 Aciertos de caché (%)                      | 99.43 |     99 |      95 | VERDE
 Transacción más antigua (min)              |     0 |      5 |      30 | VERDE
 Sesiones inactivas con transacción abierta |     0 |      1 |       3 | VERDE
 Sesiones bloqueadas                        |     0 |      1 |       5 | VERDE
 Tuplas muertas, peor tabla (%)             |   0.0 |     20 |      40 | VERDE
 Edad de XID respecto de freeze_max_age (%) |   0.0 |     50 |      80 | VERDE
 Horas desde el último respaldo OK          |  9999 |     26 |      48 | ROJO
```

Métrica	Valor	Alerta / crítico	Estado
Conexiones en uso (%)	7.0	70 / 90	VERDE
Aciertos de caché (%)	99.43	99 / 95	VERDE
Transacción más antigua (min)	0	5 / 30	VERDE
Sesiones inactivas con transacción abierta	0	1 / 3	VERDE
Sesiones bloqueadas	0	1 / 5	VERDE
Tuplas muertas, peor tabla (%)	0.0	20 / 40	VERDE
Edad de XID respecto de freeze_max_age (%)	0.0	50 / 80	VERDE
Horas desde el último respaldo OK	9999	26 / 48	ROJO

Figura 10. *Tablero de salud de Food Store (primera medición).*

El indicador en rojo no es un error del tablero: es su primer hallazgo. En ese momento ya existían archivos de respaldo en el disco, pero el script todavía no registraba su ejecución en la bitácora, de modo que **no había evidencia** de ningún respaldo exitoso. Para el monitoreo, un respaldo que no deja constancia equivale a un respaldo que no se hizo. Después de incorporar el registro en la bitácora (requisito R8 de la sección 3.1) y ejecutar el respaldo, el indicador pasó a verde:

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
              metrica               | valor | semaforo
------------------------------------+-------+----------
 Horas desde el último respaldo OK  |   0.0 | VERDE
```

Los umbrales de 26 y 48 horas para el respaldo diario no son arbitrarios: 26 horas da un margen de dos horas sobre la frecuencia diaria antes de alertar; 48 horas significa que se perdieron dos respaldos consecutivos y el RPO del respaldo lógico ya no se cumple.

5.4. El registro del servidor (log) como fuente de monitoreo

Algunos eventos no quedan en las vistas de estadísticas, sino en el archivo de registro del servidor. La Unidad 5 configuró el registro con fines de auditoría; para el monitoreo se agregan los parámetros siguientes:

Archivo 07_config_monitoreo.sql

```
-- ============================================================
-- Unidad 6 · 07_config_monitoreo.sql  (superusuario)
-- Parámetros de registro (log) que alimentan el monitoreo
-- ============================================================
ALTER SYSTEM SET log_min_duration_statement   = '500ms'; -- consultas lentas
ALTER SYSTEM SET log_autovacuum_min_duration  = '1s';    -- autovacuum que tarda
ALTER SYSTEM SET log_lock_waits               = on;      -- esperas por bloqueo > deadlock_timeout
ALTER SYSTEM SET log_checkpoints              = on;      -- frecuencia y costo de checkpoints
ALTER SYSTEM SET log_temp_files               = '10MB';  -- ordenamientos que van a disco
ALTER SYSTEM SET idle_in_transaction_session_timeout = '10min'; -- corta sesiones olvidadas
SELECT pg_reload_conf();


-- Verificación: valor vigente y origen de cada parámetro
SELECT name, setting, unit, source
FROM pg_settings
WHERE name IN ('log_min_duration_statement','log_autovacuum_min_duration',
               'log_lock_waits','log_checkpoints','log_temp_files',
               'idle_in_transaction_session_timeout');
```

Salida obtenida (PostgreSQL 16.15, base Food Store poblada)

```
                 name                  | setting | unit |       source
---------------------------------------+---------+------+--------------------
 idle_in_transaction_session_timeout | 600000  | ms   | configuration file
 log_autovacuum_min_duration         | 1000    | ms   | configuration file
 log_checkpoints                     | on      |      | configuration file
 log_lock_waits                      | on      |      | configuration file
 log_min_duration_statement          | 500     | ms   | configuration file
 log_temp_files                      | 10240   | kB   | configuration file
```

`ALTER SYSTEM` escribe los valores en el archivo `postgresql.auto.conf`, que el motor lee después del archivo principal; `pg_reload_conf()` los aplica sin reiniciar. La columna `source` de `pg_settings` confirma de dónde proviene el valor vigente de cada parámetro.

5.5. Detección temprana: tendencias, no fotografías

Una medición aislada informa el estado actual; una serie de mediciones informa hacia dónde va la base. Un disco que crece 500 MB por semana y tiene 5 GB libres se llena en diez semanas: esa fecha se conoce con anticipación solamente si el tamaño se registra en forma periódica. La extensión natural del tablero es guardar una fotografía diaria de `v_salud` y de los tamaños en una tabla histórica del esquema `mantenimiento`, programada con `pg_cron`. Esta extensión forma parte de la consigna del trabajo práctico de laboratorio.

> **Herramientas externas de monitoreo**
>
> En producción, las métricas de PostgreSQL suelen exportarse a sistemas de monitoreo de propósito general (por ejemplo, Prometheus con su exportador para PostgreSQL y paneles de Grafana, ambos de código abierto), que conservan las series, grafican tendencias y envían alertas. Esas herramientas leen exactamente las mismas vistas `pg_stat_*` que se estudian en

esta sección: comprender las vistas es el requisito para configurar y leer correctamente cualquiera de ellas.

6. La IA como motor primario del mantenimiento

6.1. El flujo de trabajo

En esta unidad, la IA participa en dos tipos de tarea: **generar** scripts de respaldo y mantenimiento a partir de una especificación, y **asistir en el diagnóstico** a partir de métricas y registros. En ambos casos, la cátedra exige el mismo flujo, en el que cada paso deja evidencia en el repositorio:

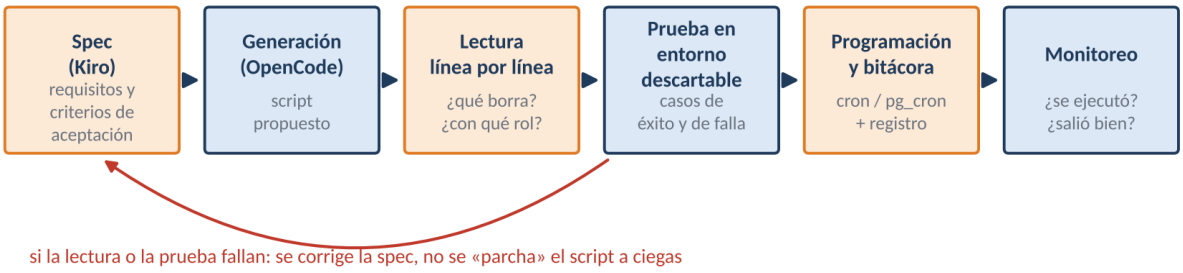


Figura 11. *Flujo de generación de scripts de mantenimiento con IA.*

6.2. La especificación en Kiro

La especificación es el documento que se escribe antes del código y contra el cual se revisa el resultado. En Kiro, los requisitos se expresan como criterios de aceptación verificables. El fragmento siguiente corresponde a la especificación del script de respaldo de la sección 3:

Fragmento de la especificación (requirements.md)

```
# Requisito: respaldo lógico diario de food_store

## Historia
Como administrador de Food Store necesito un respaldo lógico diario verificable
para cumplir el RPO de 24 h del respaldo lógico y poder restaurar tablas individuales.

## Criterios de aceptación
1. CUANDO el script se ejecute, EL SISTEMA DEBE conectarse con el rol mant_backup
   y NO DEBE contener contraseñas en el código.
2. CUANDO cualquier comando falle, EL SISTEMA DEBE detenerse, eliminar el archivo
   parcial y registrar estado ERROR en mantenimiento.bitacora.
3. CUANDO el respaldo termine, EL SISTEMA DEBE verificar el archivo con
   pg_restore --list antes de darle el nombre definitivo.
4. EL SISTEMA DEBE eliminar únicamente archivos food_store_*.dump(.sha256) de la
   carpeta de destino con antigüedad mayor a RETENCION_DIAS.
5. SI la carpeta de destino es vacía o "/", ENTONCES EL SISTEMA DEBE abortar sin
   borrar nada.

## Casos de prueba obligatorios (entorno descartable)
- Ejecución exitosa: archivo .dump + .sha256 + fila OK en la bitácora.
- Servidor detenido: código de salida distinto de 0, sin archivo .dump.
- DESTINO="/": aborta antes de cualquier operación.
```

6.3. Generación con OpenCode y revisión humana

Con la especificación cargada en el repositorio, se solicita el script a OpenCode. El ejemplo siguiente es **ilustrativo**: reúne en una sola respuesta los errores que la cátedra encuentra con mayor frecuencia en los scripts de mantenimiento generados sin especificación o revisados sin atención. El objetivo del ejercicio es encontrarlos.

Prompt entregado a OpenCode (sin especificación)

```
"Generar un script bash de mantenimiento completo para la base food_store:
 respaldo diario, limpieza de respaldos viejos y optimización de tablas."
```

Script propuesto por la IA (ejemplo con errores típicos)

```
#!/bin/bash
export PGPASSWORD="Adm1n2026!"
DIR=$BACKUP_DIR
pg_dump -U postgres food_store > $DIR/food_store.sql
rm -rf $DIR/*.sql.old
find $DIR -mtime +7 -exec rm -rf {} \;
psql -U postgres -d food_store -c "VACUUM FULL;"
psql -U postgres -d food_store -c "REINDEX DATABASE food_store;"
echo "Backup completado"
```

Línea	Problema	Corrección
2	Contraseña en texto plano dentro del script versionado	`.pgpass` con permisos 600 (requisito R2)
3	Si `BACKUP_DIR` no está definida, `DIR` queda vacía: la línea 5 se convierte en `rm -rf /*.sql.old` (opera sobre la raíz del sistema) y la línea 6 en `find -mtime +7 -exec rm -rf`, que recorre la carpeta actual	`set -u` y validación explícita del destino (R7)
4	Superusuario `postgres` para respaldar	Rol `mant_backup` con `pg_read_all_data` (R1)
4	Formato texto plano, siempre con el mismo nombre: cada ejecución sobrescribe el respaldo anterior	`-Fc` y nombre con fecha y hora
4	Sin control de errores: si `pg_dump` falla, el archivo queda vacío y el script continúa	`set -euo pipefail`, archivo `.parcial` y `pg_restore --list` (R3-R5)
6	`find` sin `-maxdepth` ni patrón de nombre: borra cualquier archivo o carpeta con más de 7 días, incluidas subcarpetas	Patrón explícito y `-type f -maxdepth 1` (R7)
7	`VACUUM FULL` de toda la base cada noche: bloqueo exclusivo de todas las tablas (sección 4.2)	`VACUUM (ANALYZE)` sobre las tablas operativas
8	`REINDEX DATABASE` diario sin `CONCURRENTLY`: bloquea escrituras y no responde a ninguna necesidad medida	Reindexar solo cuando una medición lo justifique
9	Informa éxito siempre, haya fallado o no; no deja registro consultable	Fila OK o ERROR en la bitácora (R8)

La corrección humana, documentada

La propuesta se rechaza completa y se vuelve a solicitar **con la especificación de la sección 6.2 como contexto**. La decisión se registra en la bitácora de IA del proyecto: «Script inicial rechazado: 9 defectos, 3 de ellos con posibilidad de pérdida de datos (líneas 3, 4 y 6). Se regeneró a partir de requirements.md; versión final revisada línea por línea y probada con los tres casos obligatorios». El script resultante es el de la sección 3.2.

El defecto de la línea 3 merece atención especial: ningún asistente lo «ve» si no se le pregunta explícitamente qué ocurre cuando una variable no está definida. Esa pregunta es responsabilidad de quien revisa.

6.4. Diagnóstico asistido: hipótesis contra verificación

La IA es útil para interpretar métricas y registros, siempre que se respeten dos reglas. La primera es la frontera de privacidad de la Unidad 5: al asistente se le envían **métricas y estructuras** (salidas de `pg_stat_*`, planes de ejecución, mensajes de error, definiciones de tablas), nunca filas con datos personales. La segunda es que toda explicación de la IA es una **hipótesis** que se verifica en el motor antes de actuar.

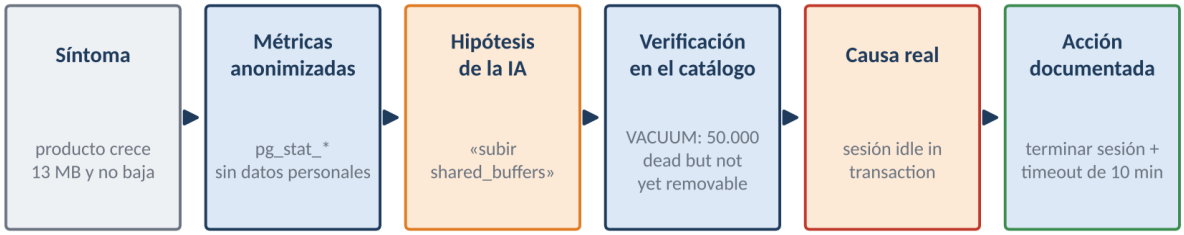


Figura 12. *Caso de diagnóstico: la hipótesis de la IA se descarta con una verificación en el catálogo.*

Caso trabajado sobre Food Store

Síntoma: la tabla `producto` ocupa 13 MB, el doble de lo esperado, y no reduce su tamaño después de varios ciclos de autovacuum.

Hipótesis de la IA (a partir de la salida de `pg_stat_user_tables`): «la memoria compartida es insuficiente; aumentar `shared_buffers` y programar `VACUUM FULL` nocturno».

Verificación: `VACUUM (VERBOSE) producto` informa `50000 are dead but not yet removable`. La cantidad de memoria no explica tuplas que no se pueden eliminar; una transacción abierta sí. La consulta a `pg_stat_activity` encuentra una sesión `idle in transaction` (sección 4.4).

Acción: se termina la sesión, se ejecuta `VACUUM` y se configura `idle_in_transaction_session_timeout`. La hipótesis de la IA se descarta y el motivo queda documentado.

6.5. Lista de control para revisar un script de mantenimiento

Antes de programar cualquier script de mantenimiento, generado o no con IA, se responden por escrito estas preguntas:

1. ¿Con qué rol se conecta y es el de menor privilegio posible?
2. ¿Hay alguna contraseña, token o dato personal en el código?
3. ¿Qué ocurre si una variable no está definida o un comando falla en la mitad?
4. ¿Qué borra, exactamente? ¿Puede borrar algo fuera de su patrón y su carpeta?
5. ¿Toma algún bloqueo exclusivo? ¿Cuánto dura sobre la tabla más grande?
6. ¿Deja registro de éxito y de falla en un lugar que el monitoreo consulta?
7. ¿Se probó en un entorno descartable, incluido al menos un caso de falla?
8. ¿Puedo explicar cada línea en una defensa oral?

7. El plan de mantenimiento de Food Store

El plan de mantenimiento es el documento que consolida todas las decisiones de la unidad. Para cada tarea indica la frecuencia, la herramienta, la identidad con la que se ejecuta y la evidencia que deja. Es el producto que se entrega en el trabajo práctico y que se defiende en la instancia global del proyecto integrador.

Tarea	Frecuencia	Herramienta	Se ejecuta como	Evidencia
Archivado de WAL	Continuo	`archive_command`	Servicio de PostgreSQL	Carpeta de WAL archivados
Respaldo físico base	Semanal (domingo 01:00)	`pg_basebackup` + `pg_verifybackup`	Rol con atributo REPLICATION	Manifiesto verificado
Respaldo lógico + globales	Diario 02:00	`backup_food_store.sh` / `.ps1` + `pg_dumpall -g`	`mant_backup`	Fila en bitácora + .sha256
Archivado de pedidos	Mensual (día 1, 03:00)	`sp_archivar_pedidos(12, FALSE)`	`admin_datos` (pg_cron)	Fila en bitácora
VACUUM (ANALYZE)	Diario 03:30 + autovacuum	`pg_cron`	`admin_datos`	`cron.job_run_details`
Controles de integridad	Diario 04:00	`fn_verificar_integridad()`	`admin_datos` (pg_cron)	Fila en bitácora
amcheck de índices	Semanal (domingo 04:30)	`bt_index_check`	`admin_datos` (pg_cron)	`cron.job_run_details`
Prueba de restauración	Semanal (domingo 06:00)	`prueba_restauracion.sh`	Superusuario del servidor de pruebas	Log con tiempo medido
Revisión del tablero	Diaria (inicio de jornada)	`v_salud`	`mant_monitor`	Registro de alertas
Simulacro de desastre	Semestral	Procedimiento de la sección 8	Equipo completo	Informe post-incidente

Estructura del repositorio

Carpeta de mantenimiento del repositorio del proyecto

```
food_store/
├── schema/                         (unidades 1 a 4)
├── seguridad/                      (unidad 5: roles.sql, auditoria.sql)
└── mantenimiento/                  (unidad 6)
    ├── 01_roles_mantenimiento.sql
    ├── 02_esquema_mantenimiento.sql
    ├── 03_archivado_pedidos.sql
    ├── 04_verificar_integridad.sql
    ├── 05_monitoreo_salud.sql
    ├── 06_tareas_programadas.sql
    ├── 07_config_monitoreo.sql
    ├── backup_food_store.sh
    ├── backup_food_store.ps1
    ├── prueba_restauracion.sh
    ├── agenda_sistema_operativo.txt
    ├── specs/requirements.md       (especificación de Kiro)
    └── PLAN_DE_MANTENIMIENTO.md    (tabla de esta sección + decisiones)
```

8. Simulacro de desastre e informe post-incidente

Un plan de recuperación que solo existe en papel tiene el mismo valor que un respaldo que nunca se restauró. El **simulacro de desastre** es el ensayo completo del plan, con el reloj en marcha, en condiciones lo más parecidas posible a un incidente real. Su objetivo no es demostrar que todo funciona, sino descubrir qué falla mientras el costo de descubrirlo es bajo.

8.1. Desarrollo de un simulacro

1. **Preparación:** se define el escenario (por ejemplo, «el servidor se perdió por completo a las 10:40») y se designa un responsable de medir los tiempos. Nadie del equipo conoce el escenario de antemano.
2. **Declaración del incidente:** se anota la hora de inicio. Desde ese momento corre el RTO.
3. **Recuperación:** el equipo trabaja únicamente con el plan escrito, los respaldos y los scripts del repositorio. Todo lo que no está escrito y hay que resolver sobre la marcha se anota.
4. **Verificación:** la base recuperada se valida con los controles de integridad y con la comparación de conteos.
5. **Cierre:** se anota la hora de servicio restablecido y se calculan los datos perdidos (RPO real) y el tiempo empleado (RTO real).

8.2. El informe post-incidente

Después de cada simulacro o incidente real se redacta un informe con formato fijo. La práctica profesional recomienda que el informe sea **sin culpables**: describe qué ocurrió y qué condiciones lo permitieron, no quién se equivocó, porque el objetivo es corregir el sistema y no castigar a una persona.

Sección del informe	Contenido
Resumen	Qué ocurrió, en dos o tres oraciones

Sección del informe	Contenido
Línea de tiempo	Hora de cada evento: detección, decisiones, comandos ejecutados, restablecimiento
Impacto	RPO real (datos perdidos) y RTO real (tiempo fuera de servicio), comparados con los objetivos
Causa raíz	La condición que permitió el incidente (no la persona)
Qué funcionó	Pasos del plan que se cumplieron tal como estaban escritos
Qué falló o faltó	Pasos ausentes, ambiguos o incorrectos en el plan
Acciones	Cambios al plan, a los scripts o al monitoreo, con responsable y fecha

9. Síntesis

Esta unidad completó la capa de datos de Food Store con la dimensión que la vuelve operable en el tiempo. Los conceptos centrales son los siguientes:

- Los objetivos de recuperación (**RPO y RTO**) son decisiones de negocio que determinan la estrategia técnica: un RPO de minutos exige archivado continuo de WAL; un RTO exige conocer el tiempo real de restauración.
- El **respaldo lógico** (`pg_dump`) es flexible y selectivo; el **respaldo físico** con WAL archivado permite la **recuperación a un instante**. Los roles se respaldan aparte, con `pg_dumpall --globals-only`.
- Un respaldo solo está probado cuando se **restauró**. La prueba de restauración se automatiza y mide el tiempo.
- MVCC genera **tuplas muertas**; `VACUUM` las limpia sin bloquear; `VACUUM FULL` devuelve espacio, pero bloquea la tabla. `ANALYZE` mantiene las estadísticas que necesita el optimizador.
- Las tareas rutinarias se **programan**, se ejecutan con roles de **mínimo privilegio** y dejan **registro** en una bitácora.
- El **monitoreo** con métricas y umbrales detecta los problemas antes que el usuario; una tarea sin registro, para el monitoreo, no se ejecutó.
- La IA redacta scripts y propone hipótesis; la especificación, la lectura línea por línea, la prueba en entorno descartable y la verificación en el motor son responsabilidad del estudiante.

La Unidad 7 abre el panorama hacia otras formas de almacenamiento (bases NoSQL y nuevos modelos). Las preguntas de esta unidad (cómo se respalda, cómo se mantiene y cómo se monitorea) se van a volver a formular para cada una de esas tecnologías.

Preguntas de repaso

1. Food Store tiene un servidor réplica que recibe todos los cambios en menos de un segundo. Un compañero sostiene que, por esa razón, los respaldos diarios son innecesarios. Explique por qué el argumento es incorrecto con un ejemplo concreto del proyecto.
2. Defina RPO y RTO. ¿Qué técnica de respaldo permite cumplir un RPO de 5 minutos y por qué un respaldo lógico diario no lo permite?
3. Se restauró en un servidor nuevo el archivo generado por `pg_dump` y todos los `GRANT` fallaron. ¿Cuál es la causa más probable y cómo se previene?
4. En el ensayo de la sección 4.2, ¿por qué el tamaño de `producto` no disminuyó después de `VACUUM` y sí después de `VACUUM FULL`? ¿Por qué no conviene programar `VACUUM FULL` todas las noches?
5. Una consulta muestra `50000 are dead but not yet removable`. Indique la causa más frecuente, la consulta que permite confirmarla y el parámetro que la previene.
6. ¿Por qué el script de respaldo escribe primero un archivo `.parcial` y lo renombra al final? ¿Qué problema evita?
7. ¿Qué controló la función `fn_verificar_integridad()` que las restricciones declarativas no controlan? ¿Qué hueco de diseño de la Unidad 3 reveló?
8. ¿Por qué el respaldo se programa en el sistema operativo y el `VACUUM` en `pg_cron`?
9. Encuentre tres defectos con riesgo de pérdida de datos en el script de la sección 6.3 y explique cada uno.
10. ¿Qué información de la base se puede compartir con un asistente de IA durante un diagnóstico y cuál no? Justifique con la Unidad 5.

Glosario de términos

amcheck. Extensión de PostgreSQL que verifica la consistencia estructural de los índices B-tree.

Archivado de WAL. Copia de cada segmento completo del registro de transacciones a un almacenamiento separado, mediante `archive_command`.

Autovacuum. Proceso del motor que ejecuta `VACUUM` y `ANALYZE` automáticamente cuando una tabla supera un umbral de cambios.

Bitácora. Tabla `mantenimiento.bitacora` en la que cada tarea programada registra su inicio, fin, estado y detalle.

Congelamiento (freeze). Marca que `VACUUM` aplica a filas antiguas para que sigan siendo visibles después de que el contador de transacciones dé la vuelta.

MVCC. Control de concurrencia multiversión: cada modificación crea una nueva versión de la fila en lugar de sobrescribirla.

pg_cron. Extensión que agrega a PostgreSQL un planificador de tareas con sintaxis de cron.

pg_dump / pg_restore. Herramientas de respaldo y restauración lógica de una base.

pg_basebackup. Herramienta de respaldo físico del clúster completo.

PITR. Recuperación a un instante (Point-In-Time Recovery): restauración de un respaldo físico más la reaplicación del WAL hasta un momento elegido.

Regla 3-2-1. Tres copias de los datos, en dos medios distintos, una de ellas fuera del sitio.

RPO. Objetivo de punto de recuperación: cantidad máxima de datos, medida en tiempo, que se acepta perder.

RTO. Objetivo de tiempo de recuperación: tiempo máximo aceptable hasta restablecer el servicio.

Tupla muerta. Versión de una fila que ya no es visible para ninguna transacción y espera ser eliminada por `VACUUM`.

VACUUM FULL. Reescritura completa de una tabla que devuelve espacio al sistema operativo con bloqueo exclusivo.

WAL. Registro de escritura anticipada (Write-Ahead Log) donde el motor anota cada cambio antes de aplicarlo a los archivos de datos.

Fuentes citadas

Todas las fuentes de este apunte son de acceso libre y están publicadas bajo licencias abiertas o en el dominio público. La documentación de PostgreSQL y la extensión `pg_cron` se distribuyen bajo la Licencia PostgreSQL, una licencia permisiva que autoriza su uso, copia y distribución. Las salidas de los comandos y los gráficos fueron producidas por la cátedra para este apunte, a partir de los scripts del proyecto Food Store.

Referencia	Licencia
Blum, R. y Singh, R. (2016). Data integrity: What you read is what you wrote. En B. Beyer, C. Jones, J. Petoff y N. R. Murphy (Eds.), *Site Reliability Engineering: How Google runs production systems* (cap. 26). O'Reilly / Google. https://sre.google/sre-book/data-integrity/	CC BY-NC-ND 4.0 (lectura libre en línea)
Citus Data. (s.f.). *pg_cron: Run periodic jobs in PostgreSQL* [Software y documentación]. GitHub. https://github.com/citusdata/pg_cron	Licencia PostgreSQL
Free Software Foundation. (s.f.). *GNU Bash reference manual* (secciones «The Set Builtin» y «Bourne Shell Builtins: trap»). https://www.gnu.org/software/bash/manual/	GNU FDL
Free Software Foundation. (s.f.). *GNU Findutils manual* (acciones -mtime y -delete). https://www.gnu.org/software/findutils/manual/html_mono/find.html	GNU FDL
Ley N.º 25.326 de Protección de los Datos Personales. (2000). Boletín Oficial de la República Argentina. https://servicios.infoleg.gob.ar/infolegInternet/anexos/60000-64999/64790/texto.htm	Texto normativo oficial de acceso público
Microsoft. (s.f.). *schtasks create*. Windows Server documentation. https://learn.microsoft.com/es-es/windows-server/administration/windows-commands/schtasks-create	CC BY 4.0 (repositorio MicrosoftDocs)
PostgreSQL Global Development Group. (2025a). *Backup and restore* (cap. 26). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/backup.html	Licencia PostgreSQL

Referencia	Licencia
PostgreSQL Global Development Group. (2025b). *Continuous archiving and point-in-time recovery (PITR)*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/continuous-archiving.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025c). *pg_dump*, *pg_dumpall* y *pg_restore*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/app-pgdump.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025d). *pg_basebackup* y *pg_verifybackup*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/app-pgbasebackup.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025e). *Routine database maintenance tasks* (cap. 25: vacuuming, ANALYZE, wraparound, REINDEX). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/maintenance.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025f). *Introduction to MVCC*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/mvcc-intro.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025g). *VACUUM*, *ANALYZE* y *REINDEX*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/sql-vacuum.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025h). *amcheck*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/amcheck.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025i). *The cumulative statistics system*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/monitoring-stats.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025j). *pg_stat_statements*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/pgstatstatements.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025k). *Error reporting and logging* y *Client connection defaults* (idle_in_transaction_session_timeout). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/runtime-config-logging.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025l). *Predefined roles* (pg_read_all_data, pg_monitor). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/predefined-roles.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2025m). *Data checksums* y *pg_checksums*. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/checksums.html	Licencia PostgreSQL
PostgreSQL Global Development Group. (2024). *Predefined roles* (pg_maintain) y *pg_basebackup --incremental*. PostgreSQL 17 documentation. https://www.postgresql.org/docs/17/predefined-roles.html	Licencia PostgreSQL
Ruggiero, P. y Heckathorn, M. A. (2012). *Data backup options*. United States Computer Emergency Readiness Team (US-CERT / CISA). https://www.cisa.gov/sites/default/files/publications/data_backup_options.pdf	Obra del gobierno de EE. UU. (dominio público)
Swanson, M., Bowen, P., Phillips, A. W., Gallup, D. y Lynes, D. (2010). *Contingency planning guide for federal information systems* (NIST Special Publication 800-34 Rev. 1). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-34r1	Obra del gobierno de EE. UU. (dominio público)